OnBuch+

SIGNED COURSE • PREMIUM

ICT • UPPER SIXTH | UPPER SIXTH — ICT

CHAPTER 23

USING AND RDBMS

SUGGESTED TIME: 5 PERIODS

IN THIS CHAPTER

This chapter introduces the practical use of a Relational Database Management System: designing tables, linking them through relationships, interrogating data with queries, and presenting results through forms and reports. Learners progressively build a complete, exam-ready database application around a familiar Cameroonian scenario.

- By the end of this chapter I can define the key terms of a relational database: table, record, field, primary key, foreign key.
- By the end of this chapter I can create tables in an RDBMS, choosing appropriate data types and field properties for each attribute.
- By the end of this chapter I can set primary keys and create one-to-many relationships with referential integrity between tables.
- By the end of this chapter I can build select queries using criteria, sorting and calculated fields to extract specific information.
- By the end of this chapter I can write queries involving multiple tables, aggregate functions (COUNT, SUM, AVG, MIN, MAX) and grouping.
- By the end of this chapter I can design user-friendly forms for data entry and navigation.

DOWNLOAD ONBUCH
Free • Google Play
AI tutor, past papers, results

GO ONBUCH+
All signed courses, unlimited AI tutor, PDF sheets — OnBuch+ tab in the app

THIS SIGNED COURSE CONTAINS

COURSE complete, illustrated, step by step

METHODS and worked examples

EXERCISES exam style + solutions

SUMMARY SHEET the essentials to remember

CONTENTS

1. Section 1: Creating Tables and Relationships (Lesson 68)
2. Section 2: Queries 1 — Selecting and Filtering Data (Lesson 69)
3. Section 3: Queries 2 — Multi-table and Summary Queries (Lesson 70)
4. Section 4: Forms, Reports and Integration (Lessons 71–72)
5. Integration activity
6. Methods and worked examples
7. Exercises
8. Solutions to the exercises
9. Summary sheet

CHAPTER OBJECTIVES

- By the end of this chapter I can define the key terms of a relational database: table, record, field, primary key, foreign key.
- By the end of this chapter I can create tables in an RDBMS, choosing appropriate data types and field properties for each attribute.
- By the end of this chapter I can set primary keys and create one-to-many relationships with referential integrity between tables.
- By the end of this chapter I can build select queries using criteria, sorting and calculated fields to extract specific information.
- By the end of this chapter I can write queries involving multiple tables, aggregate functions (COUNT, SUM, AVG, MIN, MAX) and grouping.
- By the end of this chapter I can design user-friendly forms for data entry and navigation.
- By the end of this chapter I can generate formatted reports that group, sort and summarise data for printing.
- By the end of this chapter I can integrate tables, queries, forms and reports into a small but complete database application.
- By the end of this chapter I can answer GCE-style structured questions on database design and justify design choices.

PREREQUISITES

- Notion of data, information and data processing (Lower Sixth, introduction to information systems)
- Basic file management and use of a keyboard/mouse in a GUI environment
- Concept of a database and DBMS seen in Lower Sixth (definition, advantages over flat files)
- Elementary logic: comparison operators (=, <, >, <>) and Boolean connectors AND/OR
- Familiarity with spreadsheets (rows, columns, cells) as a bridge to tables

1 SECTION 1: CREATING TABLES AND RELATIONSHIPS (LESSON 68)

Every term, the principal of a secondary school in Bamenda struggles to produce class lists, fee payment records and report cards from piles of paper registers; names are misspelt, students are counted twice, and finding who has not paid fees takes days. Meanwhile, the MTN Mobile Money agent next door retrieves any customer's full transaction history in seconds. The difference is not the computer — it is that one uses a well-designed relational database and the other does not. How are such databases structured so that information is stored once, linked intelligently, and retrieved instantly? This chapter takes you inside a Relational Database Management System (RDBMS) and teaches you to build one yourself.

1.1 Key Vocabulary and Concepts

Before we design anything, we must speak the language of databases precisely.

DEFINITION **Database, Table, Record, Field**

A **database** is an organised collection of structured data stored electronically. In a relational database, data is stored in **tables** (also called entities). Each table represents a single type of object (e.g. Student, Class, Payment).

- A **record** (or tuple, row) is one complete set of information about a single instance of the entity (e.g. one student).
- A **field** (or attribute, column) is a single piece of information about the entity (e.g. Student's surname, date of birth).

The **structure** (or schema) of a database is the definition of its tables, fields, data types and relationships — the empty skeleton. The **data** are the actual values that fill that skeleton.

KEY POINTS **Data vs Structure**

- Structure (metadata): Table names, field names, data types, primary keys, relationships. Defined once in Design View.
- Data: The actual records entered later in Datasheet View. Changing structure after data entry is risky and often requires conversion.

1.2 From a Real Situation to Entities: The School-Fees Scenario

Let us analyse a concrete situation: "A school needs to track students, the classes they belong to, and the fee payments they make."

We identify the **entities** (objects about which we store data):

1. **STUDENT** — each student has a matricule, names, date of birth, gender, and belongs to one class.
2. **CLASS** — each class has a code (e.g. L6A, U6S), a name (e.g. Lower Sixth Arts, Upper Sixth Science), and a form teacher.
3. **PAYMENT** — each payment records which student paid, the amount, the date, the term (Term 1, 2, 3), and the payment method (Cash, Mobile Money, Bank).

EXEMPLE **Listing Attributes for Each Entity**

Entity	Attributes (candidate fields)
STUDENT	Matricule (unique ID), Surname, FirstName, Gender, DOB, ClassCode (link to CLASS)
CLASS	ClassCode (unique ID), ClassName, FormTeacher
PAYMENT	PaymentID (unique ID), Matricule (link to STUDENT), Amount, PaymentDate, Term, Method

Notice that `ClassCode` appears in both STUDENT and CLASS, and `Matricule` appears in both STUDENT and PAYMENT. These are **foreign keys** — the glue that will link the tables.

1.3 Data Types in a Typical RDBMS

Choosing the correct data type for each field is crucial: it determines what values can be stored, how much space they occupy, and what operations (sorting, calculation, filtering) are possible. The table below summarises the most common types in Microsoft Access and LibreOffice Base.

Data Type	Typical Use	Example from Scenario
Short Text (Text)	Names, codes, short strings (up to 255 chars)	Surname, FirstName, ClassCode, Matricule (if text-based)
Long Text (Memo)	Long descriptions, notes	Remarks on a payment
Number (Integer, Long Integer, Double)	Numeric values for calculations	Amount (use Currency instead), Age (calculated)
Currency	Monetary values (fixed 4 decimal places, no rounding errors)	Amount
Date/Time	Dates and times	DOB, PaymentDate
Yes/No (Boolean)	True/False, On/Off	IsActive (student still enrolled), FeeCleared
AutoNumber	Auto-generated unique identifier (Long Integer)	PaymentID, Matricule (if system-generated)

WARNING **Choosing Between Number and Currency**

Never use `Number` (Single/Double) for money — floating-point arithmetic causes rounding errors (e.g. 1000 - 999.99 = 0.0100000000002). Always use `Currency` for financial fields.

WARNING **AutoNumber vs Natural Key**

An `AutoNumber` is a surrogate key generated by the system. A natural key (e.g. Matricule) comes from the real world. Both can serve as primary keys; see the next subsection.

1.4 Field Properties

Beyond the data type, each field has properties that control how data is entered, displayed and validated. The most important ones are:

- **Field Size:** For Text, maximum characters (e.g. 20 for Matricule). For Number, the subtype (Byte, Integer, Long Integer, Double).
- **Format:** How the value is displayed (e.g. `dd/mm/yyyy` for dates, `#,##0.00` for currency). Does not change stored value.
- **Input Mask:** A template that forces the user to enter data in a specific pattern (e.g. `00/00/0000` for dates, `>L<??????????????` for capitalised names).
- **Default Value:** A value automatically inserted for new records (e.g. `Date()` for PaymentDate, `False` for Yes/No).
- **Validation Rule:** An expression that must evaluate to True for the record to be accepted (e.g. `Amount > 0`, `PaymentDate <= Date()`).
- **Validation Text:** The message shown when the validation rule fails (e.g. ``` ``Amount must be positive.'' ```).
- **Required:** Yes/No — whether the field must contain a value (cannot be Null). Primary key fields are always Required.

EXEMPLE **Setting Properties for the PAYMENT Table**

Field	Property	Setting
Amount	Data Type	Currency
Amount	Validation Rule	> 0
Amount	Validation Text	Amount must be greater than zero.
PaymentDate	Data Type	Date/Time
PaymentDate	Default Value	=Date()
PaymentDate	Validation Rule	<= Date()
PaymentDate	Validation Text	Payment date cannot be in the future.
Term	Data Type	Short Text
Term	Validation Rule	In ("Term 1","Term 2","Term 3")
Method	Data Type	Short Text
Method	Input Mask	>L<??????????????

1.5 Primary Keys

DEFINITION **Primary Key**

A **primary key** (PK) is a field (or a combination of fields) whose value uniquely identifies each record in a table. A primary key value can never be `Null` (empty) and can never be duplicated. A table can have only one primary key (which may be composite).

PROPERTY **Criteria for a Good Primary Key**

A good primary key must be:

1. Unique: No two records share the same value.
2. Stable: The value rarely or never changes (e.g. a matricule does not change; a surname might).
3. Minimal: No subset of the key is also unique (for composite keys).
4. Non-null: Every record must have a value.

WARNING **Common Mistake: Using a Name as Primary Key**

Never use a person's name (Surname + FirstName) as a primary key. Names are nei-

ther unique nor stable: two students can share the same name, and a student may change their name (e.g. after marriage). Use a `Matricule` (assigned by the school) or an `AutoNumber` instead.

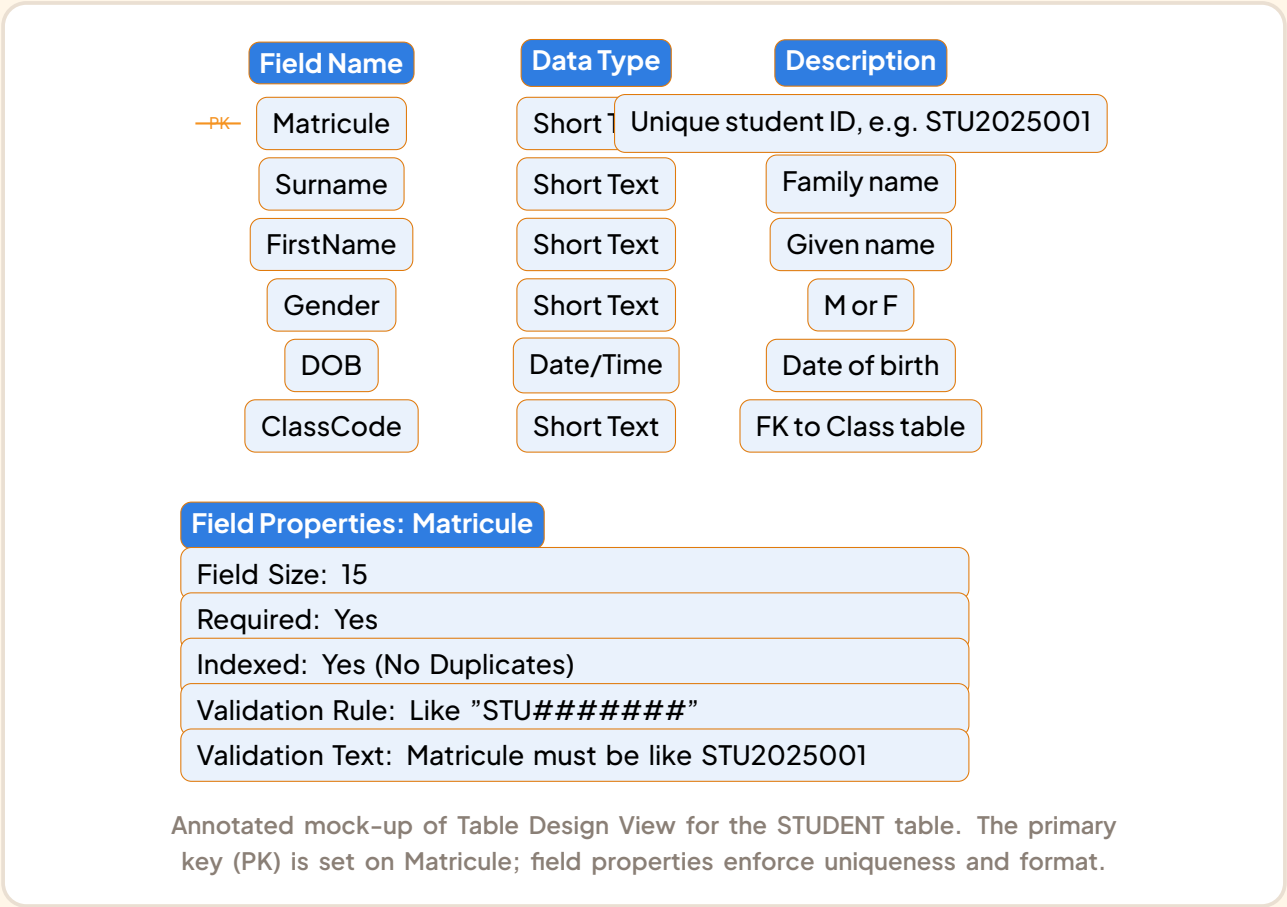
KEY POINTS **AutoNumber vs Natural Key**

- AutoNumber: System-generated, meaningless, never changes, guaranteed unique. Ideal when no stable natural key exists.
- Natural Key (e.g. Matricule): Meaningful to users, already exists in the real world. Must be guaranteed unique and stable by the organisation.
- GCE Tip: In exams, if a scenario gives a unique ID (matricule, ISBN, NIN), use it as PK. If not, create an AutoNumber field named `ID` or `EntityID`.

1.6 Step-by-Step Creation of a Table in Design View

METHOD **Designing a Table in Design View**

1. Open the RDBMS (Access / Base) and create a new blank database. Save it with a meaningful name (e.g. `SchoolFees.accdb`).
2. Choose Create → Table Design (or Table → Design View).
3. For each field:
   - Enter Field Name (no spaces preferred: `FirstName`, not `First Name`).
   - Select Data Type from the drop-down.
   - Enter an optional Description (appears in status bar during data entry).
   - Set Field Properties in the lower pane (Field Size, Format, Validation Rule, etc.).
4. Select the field(s) that form the primary key → click Primary Key toolbar button (key icon). A key symbol appears next to the field name.
5. Save the table (`Ctrl+S`) with a singular, PascalCase name: `Student`, `Class`, `Payment`.
6. Switch to Datasheet View to enter data.



Annotated mock-up of Table Design View for the STUDENT table. The primary key (PK) is set on Matricule; field properties enforce uniqueness and format.

1.7 Entering and Editing Records in Datasheet View

Once the structure is saved, switch to **Datasheet View** (spreadsheet-like grid). Each row is a new record; each column is a field.

- **Entering data:** Click in a cell, type, press `Tab` or `Enter` to move to next field. The record is saved automatically when you leave the row.
- **Editing:** Click the field, modify, leave the row.
- **Deleting a record:** Click the row selector (leftmost grey box) → `Delete` key.
- **Sorting:** Click the column header → **Ascending / Descending** (temporary sort).
- **Filtering:** Click the filter arrow on a column header → tick/untick values, or use **Text Filters / Number Filters** for custom criteria (e.g. Amount ≥ 50000).

WARNING **Datasheet View is Not a Spreadsheet**

Do not treat Datasheet View like Excel. You cannot insert blank rows arbitrarily, merge cells, or write formulas across rows. The order of rows is not guaranteed unless you sort. All data manipulation should be done via queries, forms, or SQL.

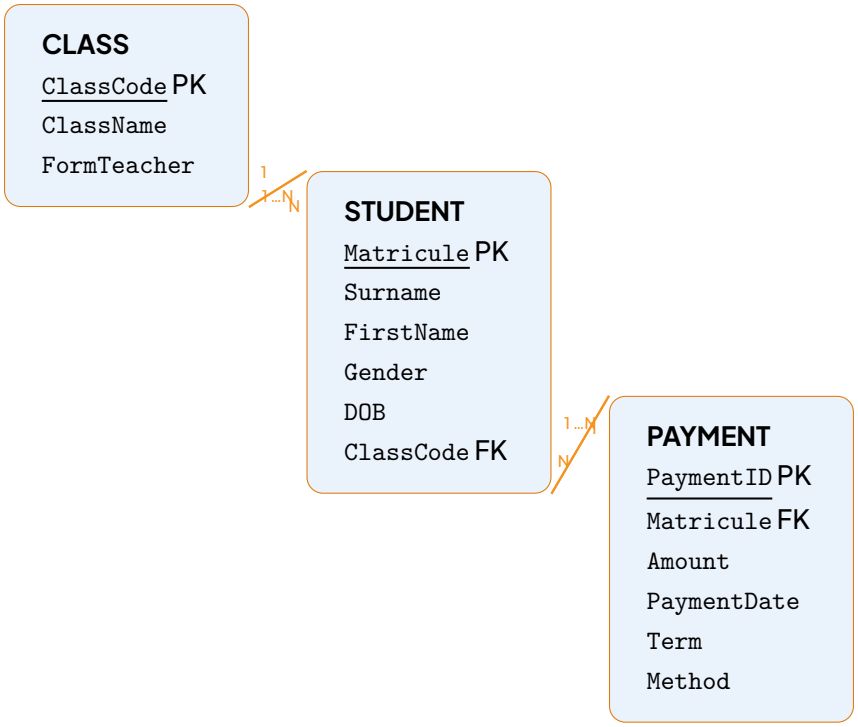
1.8 Relationships: One-to-One, One-to-Many, Many-to-Many

In a relational database, tables are linked by **relationships**. The most common is **one-to-many (1:N)**.

DEFINITION **Foreign Key**

A **foreign key** (FK) is a field (or set of fields) in one table (the child or many side) that refers to the primary key of another table (the parent or one side). The FK values must either match an existing PK value in the parent table or be `Null` (if the relationship allows optional participation).

- **One-to-Many (1:N):** One CLASS has many STUDENTs. One STUDENT makes many PAYMENTs.
  - PK of CLASS (`ClassCode`) becomes FK in STUDENT.
  - PK of STUDENT (`Matricule`) becomes FK in PAYMENT.
- **One-to-One (1:1):** Rare. Used to split a table for security or performance (e.g. `Student` and `StudentMedical` sharing the same PK).
- **Many-to-Many (M:N):** Cannot be implemented directly. Resolved by a **junction table** (also called link table or associative entity) with two FKs, each referencing one of the original tables. The PK of the junction table is often the composite of the two FKs.
  - Example: `Student` and `Subject` → junction `Enrollment` (`Matricule`, `SubjectCode`, `Grade`).



Entity-Relationship diagram (crow's-foot style). CLASS (1) to STUDENT (N) via ClassCode; STUDENT (1) to PAYMENT (N) via Matricule. Primary keys underlined, foreign keys in italics.

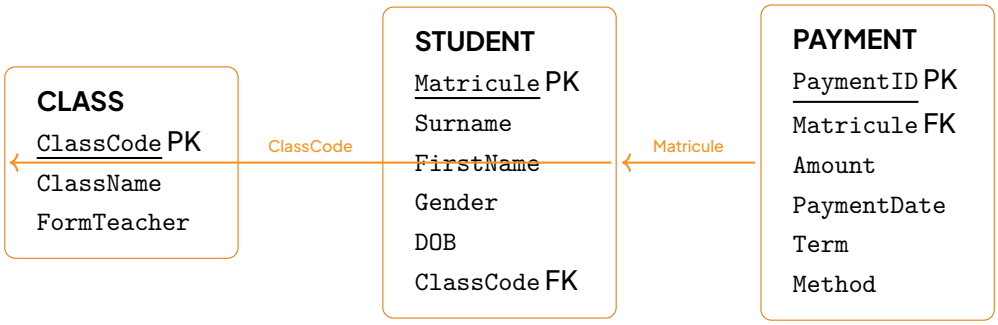


Table-schema diagram showing three tables with fields. Primary keys in bold underline, foreign keys in italics. Arrows indicate the referential links from foreign key to primary key.

1.9 Creating Relationships and Enforcing Referential Integrity

In Access / Base, relationships are created in the **Relationships Window** (**Database Tools → Relationships**).

METHOD **Creating a Relationship with Referential Integrity**

1. Open Relationships Window. Add the tables (`Class`, `Student`, `Payment`).
2. Drag the primary key field from the parent table (`Class.ClassCode`) onto the matching foreign key field in the child table (`Student.ClassCode`).
3. In the Edit Relationships dialog:
   - Verify the field names match.
   - Tick Enforce Referential Integrity (critical!).
   - Optionally tick Cascade Update Related Fields: if the PK in the parent changes (rare), the FK in all child records updates automatically.
   - Optionally tick Cascade Delete Related Records: if a parent record is deleted, all matching child records are deleted automatically. Use with extreme caution!
4. Click Create. A line appears: `1` on the parent side, ∞ (or `N`) on the child side.
5. Repeat for `Student.Matricule` → `Payment.Matricule`.
6. Save and close.

DEFINITION **Referential Integrity**

Referential Integrity is a rule enforced by the RDBMS that guarantees: every non-null foreign key value must match an existing primary key value in the referenced table. This prevents orphan records (e.g. a Payment for a Matricule that does not exist in Student).

WARNING **Cascade Delete Danger**

If you enable Cascade Delete on `Class` → `Student` and delete a class, all students in that class are permanently deleted, along with their payments. In a school system, you almost never want this. Instead, keep referential integrity without cascade delete; the system will block deletion of a class that still has students. You must first reassign or archive the students.

WARNING **Cascade Update Rarely Needed**

Primary keys should be stable (never change). If you use AutoNumber or a fixed matricule, Cascade Update is irrelevant. Only enable it if you have a business reason to change PK values (e.g. merging two school codes).

1.10 Common Design Errors and First Steps Towards Normalisation

Good design avoids three classic pitfalls. Recognising them is the first step towards **normalisation** (a GCE enrichment topic covered in university courses, but the intuition starts here).

WARNING **Repeating Groups**

Wrong: A `Student` table with fields `Payment1Date`, `Payment1Amount`, `Payment2Date`, `Payment2Amount`, …Why: Fixed number of payments, wasted space, impossible to query "all payments in Term 2". Right: Separate `Payment` table (one row per payment) linked by FK.

WARNING **Redundant Data**

Wrong: Storing `ClassName` and `FormTeacher` in every `Student` record. Why: If the form teacher changes, you must update dozens of records; risk of inconsistency. Right: Store `ClassName` and `FormTeacher` once in `Class` table; `Student` only stores `ClassCode` (FK).

WARNING **Storing Calculated Values**

Wrong: A field `TotalPaid` in `Student` table. Why: It becomes out of date as soon as a new payment is added unless you write code to maintain it. Right: Calculate it on demand with a query: `Sum(Amount) GROUP BY Matricule`. Store only raw facts; derive summaries when needed.

KEY POINTS **First Normal Form (1NF) — The Minimum for a Relational Table**

A table is in 1NF if:

- Each cell contains a single (atomic) value — no repeating groups, no arrays.
- Each record is unique (enforced by PK).
- Column order and row order are insignificant.

Our `Student`, `Class`, `Payment` design satisfies 1NF.

DID YOU KNOW? **Databases in Cameroon**

Cameroon's ANTIC (Agence Nationale des Technologies de l'Information et de la Communication) oversees the security of government databases, including the national iden-

tity register (NIN), voter registration (ELECAM), and the civil status system. Mobile Money platforms (MTN MoMo, Orange Money) process millions of daily transactions on high-availability relational clusters. The e-Government portal `www.gov.cm` relies on integrated databases for services like online tax declaration (DGI) and business registration (CFCE). Understanding RDBMS principles is therefore not just academic — it is a gateway to careers in Cameroon's growing digital economy.

1.11 Worked Example: Building the School-Fees Database from Scratch

WORKED EXAMPLE 1 **Design and Create the Three Tables with Relationships**

Statement: Using the school-fees scenario, write down the exact Design View specifications for the three tables, then describe how to create the relationships with referential integrity.
Solution:
1. TABLE: Class

Field Name	Data Type	Properties
ClassCode	Short Text	Field Size: 10; Primary Key; Required: Yes; Validation Rule: Like "L6?" Or Like "U6?"; Description: e.g. L6A, U6S
ClassName	Short Text	Field Size: 50; Required: Yes
FormTeacher	Short Text	Field Size: 50

2. TABLE: Student

Field Name	Data Type	Properties
Matricule	Short Text	Field Size: 15; Primary Key; Required: Yes; Validation Rule: Like "STU########"; Description: e.g. STU2025001
Surname	Short Text	Field Size: 30; Required: Yes
FirstName	Short Text	Field Size: 30; Required: Yes
Gender	Short Text	Field Size: 1; Validation Rule: In ("M","F"); Validation Text: Enter M or F
DOB	Date/Time	Format: Short Date; Validation Rule: < Date(); Validation Text: DOB must be in the past
ClassCode	Short Text	Field Size: 10; Required: Yes; Foreign Key (links to Class.ClassCode)

3. TABLE: Payment

Field Name	Data Type	Properties
PaymentID	AutoNumber	Primary Key; Long Integer; New Values: Increment
Matricule	Short Text	Field Size: 15; Required: Yes; Foreign Key (links to Student.Matricule)
Amount	Currency	Required: Yes; Validation Rule: > 0; Validation Text: Amount must be positive
PaymentDate	Date/Time	Format: Short Date; Default Value: =Date(); Validation Rule: <= Date(); Validation Text: Cannot be future date
Term	Short Text	Field Size: 10; Validation Rule: In ("Term 1","Term 2","Term 3")
Method	Short Text	Field Size: 15; Validation Rule: In ("Cash","Mobile Money","Bank Transfer")

4. RELATIONSHIPS

1. Open Relationships Window, add `Class`, `Student`, `Payment`.
2. Drag `Class.ClassCode` to `Student.ClassCode` → tick Enforce Referential Integrity → Create. (Line shows 1 to ∞).
3. Drag `Student.Matricule` to `Payment.Matricule` → tick Enforce Referential Integrity → Create.
4. Save and close.

Result: A relational database where each student belongs to exactly one class, each payment belongs to exactly one student, and the RDBMS prevents orphan payments or students in non-existent classes.

1.12 Summary of Key Points

KEY POINTS **Lesson 68 Essentials**

- A relational database stores data in tables (entities) linked by relationships.
- Primary Key (PK) uniquely identifies each record; choose a stable, unique field (Matricule) or AutoNumber.
- Foreign Key (FK) in the child table references the PK of the parent table.
- Data types must match the nature of the data (Currency for money, Date/Time for dates, AutoNumber for surrogate keys).
- Field properties (Validation Rule, Input Mask, Default Value, Required) protect data

quality at entry time.

- Referential Integrity ensures no orphan records; enforce it in the Relationships window.
- Cascade Update/Delete are powerful but dangerous — use only when the business logic truly requires them.
- Avoid repeating groups, redundant data, and stored calculated values — they violate First Normal Form and cause maintenance nightmares.

EXERCISE 1 **Practice: Design a Club Membership Database**

A youth club in Douala wants to manage its members, the activities they enrol in, and the attendance at each activity session.

1. Identify the entities and list their attributes.
2. Draw the ER diagram (entities, PKs, FKs, 1:N relationships).
3. Write the Design View specification for each table (field names, data types, key properties, validation rules).
4. Explain why storing `ActivityName` in the `Attendance` table would be a design error.

SOLUTION **Exercise 1**

1. Entities: MEMBER (MemberID, Surname, FirstName, Gender, DOB, Phone, JoinDate), ACTIVITY (ActivityCode, ActivityName, Description, DayOfWeek, Time), ATTENDANCE (AttendanceID, MemberID, ActivityCode, SessionDate, Present). 2. ER Diagram: MEMBER (1) —< ATTENDANCE (N) via MemberID; ACTIVITY (1) —< ATTENDANCE (N) via ActivityCode. ATTENDANCE is a junction table with composite PK (MemberID, ActivityCode, SessionDate) or surrogate AttendanceID. 3. Specifications: MemberID PK (AutoNumber or Text), ActivityCode PK (Text), AttendanceID PK (AutoNumber). FKs in ATTENDANCE. Validation: Present Yes/No, SessionDate <= Date(). 4. Redundancy: If ActivityName is stored in ATTENDANCE, changing an activity's name requires updating every attendance record. It also wastes space. The name belongs only in ACTIVITY; ATTENDANCE links via ActivityCode.

2 SECTION 2: QUERIES 1 — SELECTING AND FILTERING DATA (LESSON 69)

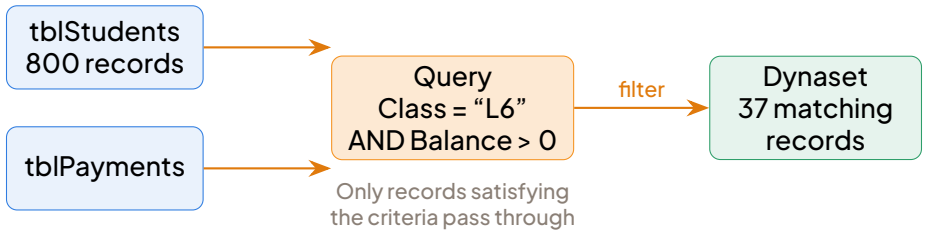
In Section 1 we built the tables of our school database, **tblStudents**, **tblClasses** and **tblPayments**, and we linked them with relationships so that the data is stored once, safely and consistently. But data locked in tables is like money locked in a safe: it only becomes useful when you can take out exactly what you need. The bursar of a college in Buea does not want to scroll through 800 student records to find "all Lower Sixth students who still owe fees". She wants to ask the question and get the answer instantly. That is precisely the job of a **query**.

2.1 What Is a Query?

DEFINITION **Query**

A **query** is a saved, reusable question asked of a database. It selects, filters, calculates or modifies data drawn from one or more tables, and presents the answer as a result called a **dynaset**.

The word dynaset ("dynamic set") is important. A query does **not** copy data into a new table. The result is a virtual table: every time you run the query, the DBMS looks at the current data in the tables and rebuilds the answer. If a student pays fees this morning, the query "students owing fees" run this afternoon will no longer show that student. This is what makes queries so powerful: they are saved once and always give an up-to-date answer.



A query acts as a filter between the tables and the dynaset (the answer).

KEY POINTS **Key ideas about queries**

- A query is an object of the database, saved with a name, just like a table.
- Running a query produces a dynaset: a live, virtual view of the data, not a copy.
- The same query can be run again and again; the answer always reflects the latest data.

2.2 Creating a Select Query in Design View

The simplest and most common type is the **select query**: it selects records and fields. In Microsoft Access (the RDBMS used in most Cameroonian school laboratories), we build it in **Design View**, using a grid at the bottom of the window.

METHOD **Building a select query**

1. Choose the table(s): Create → Query Design, then add the table(s) you need (e.g. tblStudents).
2. Drag the needed fields from the table into the Field row of the grid (only the fields you want to see).
3. Set the sort order in the Sort row (Ascending or Descending) where needed.
4. Write the criteria in the Criteria row (and the or rows if necessary).
5. Run and check: click Run (!) to see the dynaset; verify the answer makes sense.
6. Save the query with a meaningful name, e.g. qryLowerSixthDebtors.

The Design View grid has the following rows:

Row	Purpose
Field	The field displayed or used in a condition
Table	The table the field comes from
Sort	Ascending / Descending order of the result
Show	Tick to display the field, untick to hide it
Criteria	Condition(s) a record must satisfy (AND on one row)
or	Alternative conditions (OR logic, one per row)

2.3 Writing Simple Criteria

DEFINITION **Criterion**

A **criterion** (plural criteria) is a condition written in the query grid that a record must satisfy in order to appear in the dynaset.

The way a criterion is written depends on the data type of the field:

Data type	Example criterion	Meaning
Short Text	”Form 5”	class is exactly Form 5
Number / Currency	> 50000	value greater than 50 000
Date/Time	#12/01/2025#	on the date 12 January 2025
Yes/No	Yes	the box is ticked (e.g. fees paid)

Notice the delimiters: text goes in **double quotes**, dates between **hash signs #**, numbers and Yes/No stand alone.

Comparison operators. Criteria can use the operators =, <> (not equal), <, <=, >, >=, and the range operator **BETWEEN … AND**. For example, `BETWEEN 10000 AND 25000` selects values from 10 000 to 25 000 inclusive.

Wildcards with LIKE. For partial matches on text, use **LIKE** with the wildcards * (any number of characters) and ? (exactly one character):

- `LIKE "N*"` — names starting with N (Ngong, Ndi, Nkwain…);
- `LIKE "*son"` — names ending with “son”;
- `LIKE "*anga*"` — names containing “anga” anywhere;
- `LIKE "M?ndi"` — five-letter names like Mandi, Mundi.

WARNING **Text criteria must match exactly**

A criterion `"Form 5"` will not match `"Form 5"` (double space) or `"form 5"` if the data was typed differently. Most “my query returns nothing” problems come from spelling or spacing. When you are not sure of the exact text, use wildcards: `LIKE "*Form 5*"`.

2.4 Combining Criteria: AND, OR, IN, IS NULL

AND. Conditions written on the same Criteria row must all be true. Example: `"Lower Sixth"` under Class and `<25000` under FeesPaid on the same row means “Lower Sixth **AND** paid less than 25 000 FCFA”.

OR. Conditions on different rows (the or rows) are alternatives: a record appears if it satisfies any row. Example: `"Form 5"` on the Criteria row and `"Lower Sixth"` on the or row means “Form 5 **OR** Lower Sixth”.

WARNING **AND vs OR in the grid**

Writing `"Form 5"` and `"Lower Sixth"` on the same row asks for students whose class is both at once — impossible, so the dynaset is empty! OR conditions always go on separate rows.

IN. A shortcut for several ORs on one field: `IN ("Form 5", "Lower Sixth", "Upper Sixth")`.

IS NULL / IS NOT NULL. To find missing data, ordinary operators fail (nothing equals an empty field). Use `IS NULL` to find records with no value — e.g. students with **no phone number** — and `IS NOT NULL` for those that have one.

Field	Table	Sort	Show	Criteria	or
FullName	tblStudents	Ascending	✓		
Class	tblStudents		✓	"Lower Sixth"	
FeesPaid	tblStudents		✓	<25000	

same row = AND

Design View grid: Class = "Lower Sixth" AND FeesPaid < 25 000 (both criteria on the same row).

2.5 Sorting and Showing Fields

In the **Sort** row choose Ascending (A→Z, smallest first) or Descending (Z→A, largest first). You may sort on several fields: Access applies the sort from left to right in the grid, so place the most important sort field to the left. A field can be used for sorting or criteria but **hidden** from the answer by unticking its **Show** box — useful when you filter on FeesPaid but do not want to display it.

2.6 Calculated Fields

A query can compute new values that are not stored in any table. Type an expression in an empty Field cell, with a name followed by a colon:

Balance: [FeesDue]-[FeesPaid]

Field names are written in **square brackets**. The new column Balance appears in the dynaset, computed for every record. Right-click the field and open Properties to format it as **Currency** (e.g. FCFA). This is excellent database practice: the balance is never stored, so it can never disagree with the fees actually recorded.

2.7 Parameter Queries (GCE Enrichment)

A **parameter query** asks the user for a value at run time. Instead of fixing the class in the grid, write the prompt in square brackets as the criterion: `[Enter class name:]`. Each time the query runs, a dialog box appears and the user types, for example, Upper Sixth. One flexible query then replaces one query per class. This goes slightly beyond the core syllabus but is a favourite GCE enrichment question.

2.8 Saving, Re-opening and Modifying a Query

Save with Ctrl+S and a meaningful name (prefix **qry** is a common convention, e.g. **qryDebtors**). A saved query can be re-opened in Datasheet View (run it) or Design View (modify it) at any time.

WARNING **Editing a dynaset edits the table**

The dynaset is a live view of the tables. If you type over a value while looking at query results, you are changing the underlying table data itself. Treat query results with the same care as the tables.

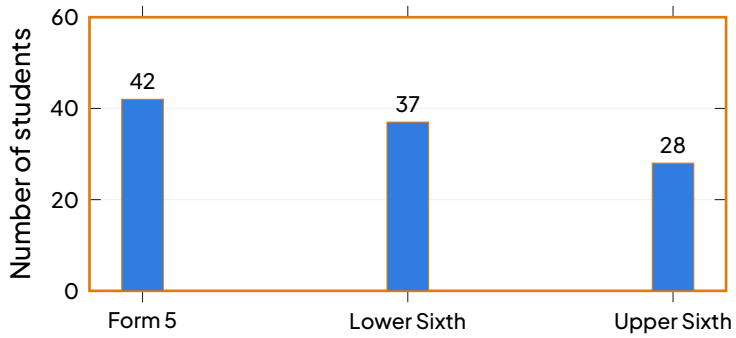
WORKED EXAMPLE 2 **Worked example: the bursar's query**

Statement. Using tblStudents(StudentID, FullName, Class, FeesDue, FeesPaid, Phone), design a query that lists, in alphabetical order, the FullName, Class and Balance of all Lower Sixth students who still owe money (Balance > 0). Describe the grid.

SOLUTION

Solution.

- Table: tblStudents.
- Fields: FullName (Sort: Ascending, Show: yes); Class (Criteria: `"Lower Sixth"`); a calculated field `Balance: [FeesDue]-[FeesPaid]` with Criteria `>0` (both criteria on the same row = AND).
- Run and check: only Lower Sixth students with a positive balance appear, sorted A–Z.
- Save as qryL6Debtors.



Visualising a query result: number of students per class extracted from tblStudents.

EXEMPLE **Exam tip**

In GCE papers, when you describe a query in words, always state three things clearly: the table(s) used, the fields displayed, and the exact criteria (with operators and values). Example: "From tblStudents, display FullName and FeesPaid where Class = "Form 5" AND

FeesPaid < 25 000 FCFA, sorted ascending on FullName."

EXERCISE 2 **Practice**

Using tblStudents above, write the criteria for each request:

1. All students whose names begin with "T".
2. Students who paid between 20 000 and 50 000 FCFA inclusive.
3. Students in Form 5 or Upper Sixth (two methods: or rows, then IN).
4. Students with no phone number recorded.
5. A calculated field `PercentPaid: [FeesPaid]/[FeesDue]*100`.

SOLUTION **Exercise 1**

1. `LIKE "T*"` on FullName.
2. `BETWEEN 20000 AND 50000` on FeesPaid.
3. `"Form 5"` on the Criteria row and `"Upper Sixth"` on the or row; or `IN ("Form 5","Upper Sixth")` on one row.
4. `IS NULL` on Phone.
5. Type in a Field cell: `PercentPaid: [FeesPaid]/[FeesDue]*100` (optionally format as Percent).

DID YOU KNOW? **Queries in real Cameroonian systems**

Every time an MTN Mobile Money agent checks "all transactions above 100 000 FCFA today", or a school in Yaoundé prints the list of candidates registered for the GCE, a select query with criteria is running behind the scenes. Mastering criteria means mastering the questions you can ask of any data.

With single-table selection and filtering now mastered, we are ready for Section 3, where queries will combine several tables through their relationships and produce summary statistics with totals, counts and averages.

3 SECTION 3: QUERIES 2 — MULTI-TABLE AND SUMMARY QUERIES (LESSON 70)

In Section 2 we learnt how to build a select query on a single table: choosing fields, sorting, and filtering with criteria. That already answers many questions, such as "list all students in Form 5". But real questions in a school, a business or a hospital rarely concern one table alone. The bursar of a college in Buea does not ask "show me the payments"; she asks "show me each payment with the student's name and class", or "what is the total amount collected per class this term?". The answers live in several tables at once: the amount is in **PAYMENT**, the name is in **STUDENT**, and the class name is in **CLASS**. This section teaches the two great powers of queries: combining several tables through **joins**, and summarising data with **aggregate functions** and **GROUP BY**.

3.1 Why Queries Need Several Tables

Recall why we split data into related tables in the first place (Section 1): to avoid repeating information and creating inconsistencies. The price of that good design is that any interesting question forces us to re-assemble the pieces. Consider our school database:

Table	Some fields
STUDENT	StudentID (PK), FullName, Sex, ClassID (FK)
CLASS	ClassID (PK), ClassName, FeesDue
PAYMENT	PaymentID (PK), StudentID (FK), Amount, PayDate, Term

The question "show each payment with the student's name and class" needs **Amount** and **PayDate** from PAYMENT, **FullName** from STUDENT, and **ClassName** from CLASS. No single table contains all three. The query must therefore join the tables, following the foreign keys we defined: PAYMENT.StudentID matches STUDENT.StudentID, and STUDENT.ClassID matches CLASS.ClassID.

DEFINITION **Join**

A **join** is the link between two tables in a query, based on matching key values (usually a primary key and the corresponding foreign key), that lets fields from both tables appear together in one result. In the query Design View, a join appears as a join line connecting the matching fields of the two tables.

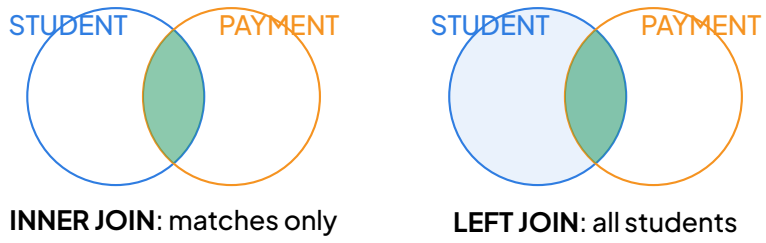
KEY POINTS **Good news: relationships do the work for you**

If you created relationships between the tables (Lesson 68), the RDBMS automatically draws the join lines as soon as you add the related tables to a query. You do not have to tell the query how STUDENT and PAYMENT are linked — it already knows.

3.2 Inner Joins and Outer Joins

Suppose STUDENT contains 5 students, but only 3 of them have made payments. What should the query "students with their payments" show?

- An **inner join** shows **only the matching records**: the 3 students who appear in both tables. A student with no payment disappears from the result; a payment with no valid student (which referential integrity normally forbids) would also disappear.
- An **outer join** keeps **all records from one side**, even when there is no match on the other side. A left outer join from STUDENT to PAYMENT shows all 5 students; the two who never paid appear with empty (Null) payment fields. This is exactly what the bursar needs to produce a **list of debtors**.



Inner join (shaded intersection only) versus left outer join (all of STUDENT plus the matching part of PAYMENT).

WARNING **Choosing the wrong join type**

By default, most RDBMS use an inner join. If your query "students and their payments" returns fewer students than expected, some students simply have no payment yet. Double-click the join line and choose the option "include ALL records from STUDENT" (a left outer join) to see everyone, including non-payers.

3.3 Building a Multi-table Select Query

METHOD **Building a multi-table query**

1. Create a new query in Design View and add all the tables needed (e.g. CLASS, STUDENT, PAYMENT). Check that the join lines appear correctly.
2. Drag the required fields from each table into the grid: e.g. ClassName (CLASS), FullName (STUDENT), Amount and PayDate (PAYMENT).
3. Apply criteria in the appropriate column(s) — criteria may mix fields from different tables, e.g. Term = "Term 1" and ClassName = "Lower Sixth Science".

4. Choose the sort order if needed, then Run the query and check the dynaset.

EXEMPLE **Fees collected per student**

Question: "List, for Term 1, each payment with the student name and class, newest first." Grid: ClassName (CLASS), FullName (STUDENT), Amount, PayDate (PAYMENT); criterion "Term 1" under Term; PayDate sorted Descending. The dynaset shows one row per payment, with the name and class repeated next to each amount — exactly what a single-table query could never produce.

3.4 Aggregate Functions and Grouping

So far every query row corresponds to one record. But management questions are summary questions: "How much did we collect in total? What is the average mark? How many students per class?" These require **aggregate functions**.

DEFINITION **Aggregate function**

An **aggregate function** is a function that computes a single value from a whole set of records. The five standard aggregates are: COUNT (number of records), SUM (total of a numeric field), AVG (average), MIN (smallest value) and MAX (largest value).

EXEMPLE **Typical uses**

SUM(Amount) = total fees collected; AVG(Mark) = average mark in a test; COUNT(StudentID) = number of students; MIN(PayDate) = the oldest payment date; MAX(PayDate) = the most recent payment.

Used alone, an aggregate produces one value for the whole table. The real power comes from grouping: computing the aggregate separately for each category.

PROPERTY **GROUP BY**

GROUP BY gathers all records that share the same value of a chosen field into one group, so that aggregate functions are computed per group, producing one result row per group. (Understanding and applying this property is examinable; no formal proof is required.)

In the Design View, grouping is activated by displaying the **Total row** (the Σ button). Each col-

umn of the grid then offers: Group By, Sum, Avg, Count, Min, Max, Where, etc.

METHOD **Summarising data**

1. Add the table(s) needed to the query.
2. Display the Total row (click Σ).
3. Set Group By on the grouping field (e.g. ClassName).
4. Choose the aggregate on the value field (e.g. Sum on Amount).
5. Add criteria if needed, then run and check the dynaset.

WORKED EXAMPLE 3 **Total fees collected per class**

Statement. Design a query that shows, for each class, the total amount of fees collected, and give the expected result for: Lower Sixth Science (3 payments of 50 000, 75 000 and 100 000 FCFA), Upper Sixth Arts (2 payments of 60 000 and 40 000 FCFA).

SOLUTION

Solution. Add CLASS, STUDENT and PAYMENT. Show the Total row. In the grid: ClassName with Total = Group By; Amount with Total = Sum. The dynaset is:

ClassName	SumOfAmount (FCFA)
Lower Sixth Science	225 000
Upper Sixth Arts	100 000

Check: 50 000 + 75 000 + 100 000 = 225 000 and 60 000 + 40 000 = 100 000. One row per class, as GROUP BY promises. Similarly, replacing Sum by Count on StudentID would give the number of students per class.

Field	ClassName	Amount
Table	CLASS	PAYMENT
Total	Group By	Sum
Sort	Ascending	
Show	Yes	Yes
Criteria		

ClassName	**SumOfAmount**
Form 5	480 000
Lower Sixth Sc.	225 000
Upper Sixth Arts	100 000

Query grid (left) for “total fees per class” and the resulting dynaset (right).

3.5 Filtering Grouped Results: the HAVING Logic

Suppose the bursar wants only the classes where the total collected exceeds 500 000 FCFA. The total does not exist until after grouping, so the filter must also be applied after grouping. This is the role of **HAVING** in SQL: WHERE filters individual records before grouping; HAVING filters groups after aggregation. In the Design View, you simply type the criterion (e.g. > 500000) in the same aggregated column (the one with Sum), and the RDBMS places it correctly in the HAVING part.

WARNING **The classic GCE trap**

A very common mistake is to put a condition on an aggregated value (e.g. total > 500 000) in the WHERE part, as if it were an ordinary field criterion. WHERE runs before any total exists, so this either fails or filters individual payments instead of class totals. Rule: criteria on raw fields → WHERE; criteria on totals, counts or averages → HAVING (criterion typed in the aggregated column).

3.6 Action Queries and Crosstab Queries (GCE Enrichment)

Select queries never change the data. **Action queries** do, and they must be used with care:

Action query	Effect
Update	Modifies values in existing records (e.g. increase all fees by 10 %).
Append	Adds records from one table into another (e.g. archive last year's payments).
Delete	Removes whole records matching a criterion (e.g. delete cancelled registrations).
Make-table	Creates a new table from the query result.

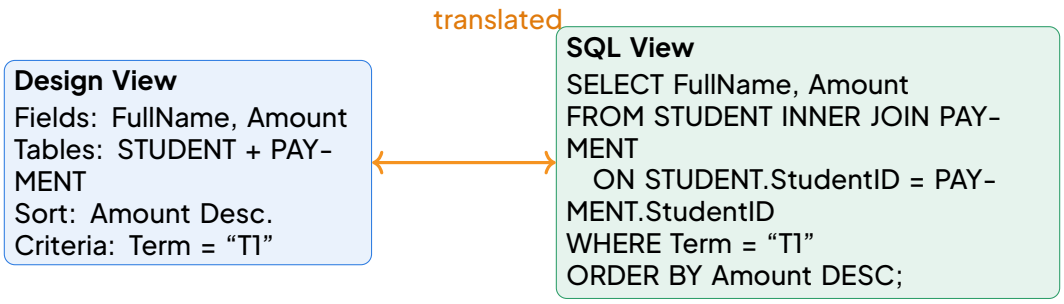
WARNING **No undo!**

An action query can modify or destroy thousands of records in one click, and the operation cannot be undone. Always back up the database and preview the selection with a select query first.

A **crosstab query** presents grouped data as a two-dimensional summary: for example, classes as rows, terms as columns, and the sum of amounts in the cells — ideal for comparing fee collection across terms at a glance.

3.7 Reading SQL: Behind the Design Grid

Every query you build in Design View is translated by the RDBMS into **SQL** (Structured Query Language). GCE questions frequently show an SQL statement and ask for its effect, or the reverse. The correspondence is direct:



Each element of the Design View maps to a clause of the SQL statement: fields → SELECT, tables and joins → FROM …JOIN, criteria → WHERE, sort → ORDER BY.

KEY POINTS **The SQL skeleton to recognise**

SELECT fields FROM table(s) with joins WHERE record criteria GROUP BY grouping field HAVING group criteria ORDER BY sort field. A grouped query such as "classes with total above 500 000 FCFA" reads: SELECT ClassName, SUM(Amount) …GROUP BY ClassName HAVING SUM(Amount) > 500000.

EXERCISE 3 **Translating both ways**

1. State in plain English the effect of: SELECT ClassName, COUNT(StudentID) FROM STUDENT INNER JOIN CLASS ON STUDENT.ClassID = CLASS.ClassID GROUP BY ClassName HAVING COUNT(StudentID) > 40.
2. Design (grid or SQL) the query: "total fees collected in Term 2, per class, classes with less than 200 000 FCFA only".

SOLUTION **Exercise 1**

1. It displays, for each class, the number of students, keeping only classes with more than 40 students. Note the HAVING: the count is an aggregate, so it is filtered after grouping.
2. Tables CLASS, STUDENT, PAYMENT; Total row shown. ClassName: Group By. Amount: Sum, with criterion < 200000 in that same column (HAVING logic). Term: Total = Where, criterion "Term 2" (a raw-field filter applied before grouping). SQL: SELECT ClassName, SUM(Amount) FROM CLASS INNER JOIN (STUDENT INNER JOIN PAYMENT ON STUDENT.StudentID = PAYMENT.StudentID) ON CLASS.ClassID = STU-

DENT.ClassID WHERE Term = “Term 2” GROUP BY ClassName HAVING SUM(Amount) < 200000.

DID YOU KNOW? **SQL everywhere**

SQL is not tied to one product: the same SELECT–FROM–WHERE logic runs the databases behind mobile money platforms, bank systems and e-government services used daily in Cameroon. Mastering the translation between an English question, a design grid and SQL is a skill valued far beyond the GCE hall.

WARNING **Exam tip**

GCE papers regularly give an SQL statement and ask “what does this query do?”, or give an English question and ask for the query design. Practise translating both ways, and always check: are the criteria on raw fields (WHERE) or on totals (HAVING)? Is an inner join enough, or must non-matching records be kept (outer join)?

We can now combine tables and summarise data — but results still appear as raw datasheets. In Section 4 we will present them professionally through **forms** for data entry and **reports** for printing, before integrating everything in a complete database project.

4 SECTION 4: FORMS, REPORTS AND INTEGRATION (LESSONS 71–72)

In Section 3 we learnt how to interrogate several tables at once and how to summarise data with aggregate queries. Queries answer our questions, but they are not the face of the application. The bursar of a school in Bamenda should never have to open a raw table or type SQL to record a fee payment. What the user sees and touches is a **form**; what the principal takes to the Parents’ Association meeting is a printed **report**. In this final section we complete the picture: forms for input, reports for output, and the **integration activity** that assembles tables, relationships, queries, forms and reports into one coherent mini-application.

4.1 Forms: User-Friendly Data Entry

Observation. Open the `STUDENT` table in Datasheet View: you see hundreds of rows and many columns. To register one new student you must scroll, type in a grid, and remember exact codes. Now imagine instead a screen showing one student at a time, with labelled boxes, a drop-down list for the class, and buttons “Next record” and “Save”. That screen is a form.

DEFINITION **Form**

A **form** is a database object used to enter, edit and display records one at a time in a user-friendly layout. A form is usually bound to a table or query (its **RecordSource** property), so whatever is typed into the form is stored in the underlying table.

The essential difference from Datasheet View is presentation and control: a table shows many records as a raw grid with no guidance, while a form shows one record (or one record plus a related list) with labels, help, choice lists and buttons. The form is the interface; the table remains the storage.

4.2 Creating a Form; Bound and Unbound Controls

A form can be created quickly with the **Form Wizard** (choose the table or query, select the fields, pick a layout such as Columnar, Tabular, Datasheet or Justified, then finish), or built and refined manually in **Layout View** (visual, drag-and-drop) or **Design View** (pixel-perfect control over every property).

Every element placed on a form is a **control**, and controls fall into two families:

- **Bound controls** are linked to a field of the underlying table or query via their **ControlSource** property. A text box bound to `StudentName` displays and edits that field. Typing in it changes the stored data.
- **Unbound controls** are not linked to any field: titles, explanatory labels, decorative lines, or calculated expressions (e.g. `= [Amount] * 1.1925` for VAT). They only display fixed or computed information.

The main controls you must know are:

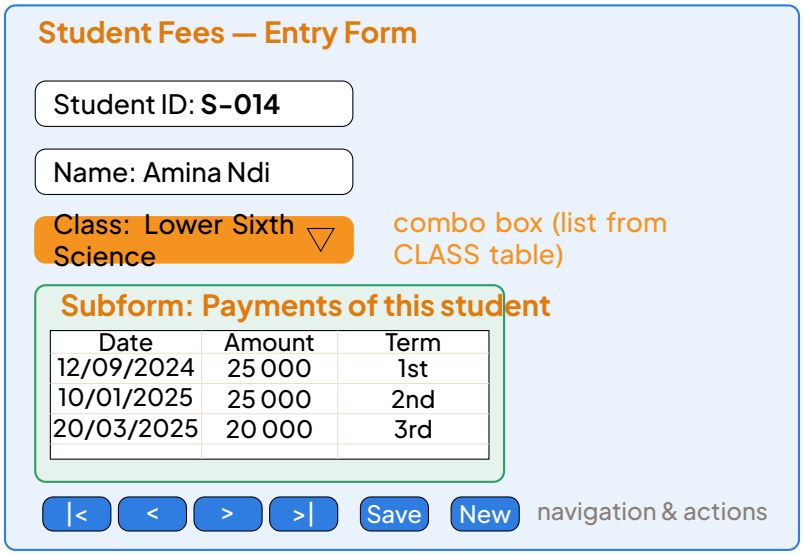
Control	Purpose on a form
Label	Fixed text: titles, field captions (unbound).
Text box	Displays/edits a field (bound) or a calculation (unbound).
Combo box	Drop-down list fed by another table or value list, e.g. choosing a `ClassID` from the `CLASS` table; prevents typing invalid values.
List box	Similar to combo box but shows multiple choices at once.
Check box	Yes/No fields, e.g. `FeesCompleted`.
Option group	Mutually exclusive choices (e.g. payment method: Cash, Mobile Money, Card).
Command button	Runs an action: save record, new record, close form, print, run macro.
Subform	Shows related records (the "many" side) inside the main form.
Navigation buttons	Built-in record navigator (First, Previous, Next, Last, New).

Each control has **properties** (Name, ControlSource, RowSource for combo boxes, Format, InputMask, ValidationRule, ValidationText, Enabled, Visible, TabStop). The **tab order** — the order in which the cursor jumps when the user presses Tab — should follow the natural order of filling

the form (name, then class, then amount), so that data entry is fast and error-free. Set it via the Tab Order dialog in Design View.

4.3 Subforms: One-to-Many on One Screen

A one-to-many relationship can be displayed on a single screen: the main form shows the "one" side and a **subform** shows the related "many" records as a small grid. Example: a `STUDENT` main form with a subform listing all that student's `PAYMENT`s. The two are linked by the common field (`StudentID`) using the subform's **LinkMasterFields** and **LinkChildFields** properties; when you navigate to another student, the subform automatically shows only his or her payments.



Mock-up of a "Student Fees" form: bound text boxes, a combo box fed by the CLASS table, a subform grid of payments, and navigation buttons.

4.4 Data Validation Through Forms

Forms are the ideal place to enforce **data validation**, because every entry passes through them:

- **Input masks** force a pattern, e.g. a Cameroonian phone number `6XX-XX-XX-XX` or a date `DD/MM/YYYY`. Set via the `InputMask` property (e.g. `00/00/0000;0;_`).
- **Combo boxes** with `Limit To List = Yes` restrict a field to values from another table (you can only pick an existing class).
- **Field-level validation rules** (on the table field or the control) reject impossible values (e.g. `>=0` for Amount, `Between #01/01/2020# And Date()` for payment date), and `Required = Yes` prevents blanks.
- **Form-level validation** in the `BeforeUpdate` event can check cross-field rules (e.g. "If payment method is Mobile Money, phone number must be filled").

WARNING **Entering data directly into tables**

A very common mistake — in practice and in exam answers — is to let users type straight into tables. This bypasses validation and produces inconsistent values: the same class recorded as "Form 5", "form5" and "F5". Such duplicates destroy grouping and totals. Always enter data through forms with combo boxes and validation rules.

DID YOU KNOW? **Cameroon context: Mobile Money validation**

In a real Cameroonian school fees system, the `PaymentMethod` field would be an option group (Cash, MTN MoMo, Orange Money). A validation rule on the `PhoneNumber` field could enforce the pattern `6[5-9]-\d\d-\d\d-\d\d` for MTN or `6[0-4]-\d\d-\d\d-\d\d` for Orange, using an input mask or a `Like` expression in the validation rule.

4.5 Reports: Formatted, Printable Output

Observation. At the end of term the principal wants, on paper, the list of students grouped by class, with the total fees collected per class and a grand total for the school, page numbers and the date. A screen form cannot do this; a query grid is not presentable. We need a report.

DEFINITION **Report**

A **report** is a database object that formats and summarises data for printing or export (PDF, paper), with grouping levels, headers, footers and totals. Unlike a form, a report is static: it displays data but does not allow editing.

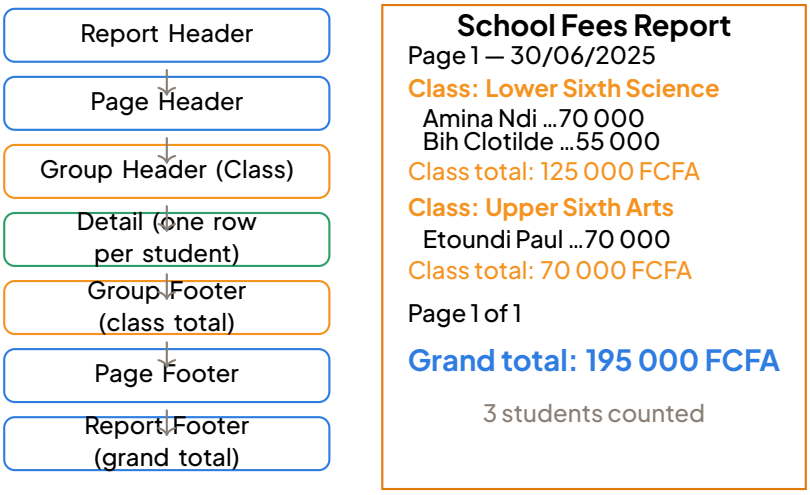
WARNING **Form vs report in the exam**

Candidates frequently confuse the two. Remember: a form is interactive and lives on screen (input/editing); a report is static and designed for printing (output). If the question mentions printing, grouping with totals, or a document for management, the answer is a report.

4.6 Creating a Report; Sections and Summary Calculations

The **Report Wizard** guides the design in steps: choose the source table or query, select the fields, add **grouping levels** (e.g. group by `ClassName`), choose **sorting** within groups (e.g. by student name), pick a layout (Stepped, Block, Outline, Align Left) and a style. The wizard also offers summary options (Sum, Count, Average, Min, Max) for numeric fields.

A finished report is organised in **sections**, each printed at a precise moment:



Sections of a grouped report (left) and a sketch of the printed fees report (right): group headers per class, detail rows, class totals in group footers, grand total in the report footer.

- **Report Header**: printed once at the very top (title, logo, date printed).
- **Page Header**: top of every page (column titles, report title repeat).
- **Group Header**: printed each time the grouping value changes (e.g. the class name). Set `GroupOn` property (Each Value, First Character, etc.).
- **Detail**: one line per record (the core data rows).
- **Group Footer**: summary per group — e.g. `=Sum([Amount])` gives the total fees per class, `=Count([StudentID])` the number of students. Place unbound text boxes here with these expressions.
- **Page Footer**: page numbering (`="Page " & [Page] & " of " & [Pages]`), confidentiality notices.
- **Report Footer**: printed once at the end — the grand total for the whole school (`=Sum([Amount])` over all records).

WORKED EXAMPLE 4 **Designing the fees report**

Statement. The bursar wants a printable report showing, for each class, the students who paid, the amount each paid, the class subtotal, and the school's grand total, sorted alphabetically within each class. Describe how to produce it.

SOLUTION

Solution.

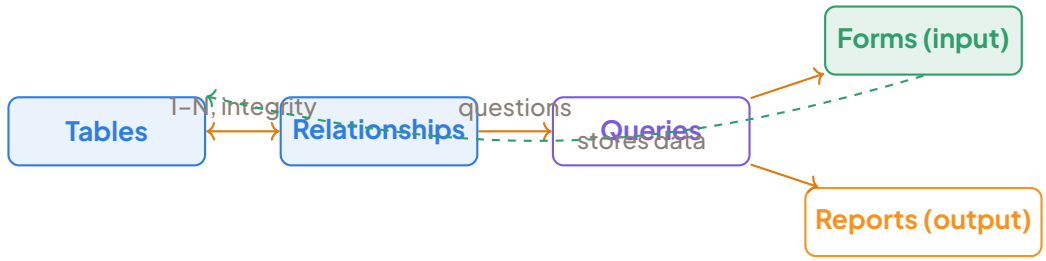
1. Create a query joining `CLASS`, `STUDENT` and `PAYMENT`, returning `ClassName`, `StudentName`, `Amount` (this is a multi-table query as in Section 3). Save as `qryFeesReportSource`.

2. Open the Report Wizard on this query; select the three fields.
3. Add a grouping level on `ClassName`; sort the detail by `StudentName` ascending.
4. In the summary options, request `Sum` of `Amount`; the wizard places it in the Group Footer and the grand total in the Report Footer.
5. Choose a Stepped layout; in Design View add the title in the Report Header and `="Page " & [Page] & " of " & [Pages]` in the Page Footer.
6. Set the report's `RecordSource` to `qryFeesReportSource` and verify print preview.

The printed result matches the sketch above: one block per class with its subtotal, and the grand total on the last page.

4.7 Integration Activity: Assembling the Mini-Application

The **integration activity** (Lesson 72) asks you to build a complete, working mini-application — for example a "School Fees Management" system — from an empty database to a tested product. The architecture is:



Architecture of the integrated application: tables linked by relationships feed the queries; queries feed the forms (input) and the reports (output); forms write validated data back into the tables.

METHOD **Building the application**

1. Design the tables and keys: identify entities (`STUDENT`, `CLASS`, `PAYMENT`), fields, data types and primary keys.
2. Set the relationships with referential integrity (`CLASS` 1–N `STUDENT`, `STUDENT` 1–N `PAYMENT`). Enable cascade update where appropriate.
3. Create a query for each question the users ask (fees per student, totals per class, debtors list, termly summary).
4. Build the forms for data entry, with combo boxes, validation rules, input masks and

subforms.

5. Build the reports for output, with grouping, sorting, subtotals and grand totals.
6. Test with realistic sample data: enter several students and payments across multiple classes and terms, check that totals on the report match hand calculations, and try to enter wrong data to confirm validation rejects it.

EXPERIMENT **Guided integration: "School Fees" mini-application**

Working in pairs: (1) create the three tables and the two relationships with referential integrity; (2) write the query "total paid per student"; (3) build the Student Fees form with a payment subform and a class combo box; (4) build the grouped fees report with class subtotals and a grand total; (5) enter five realistic students with payments across two classes; (6) verify the report's grand total equals the sum computed by hand, and try entering a payment for a non-existent class to confirm the combo box prevents it.

4.8 Good Practice and Exam Readiness

KEY POINTS **Professional habits**

- Name objects consistently: e.g. `tblStudent`, `qryFeesPerClass`, `frmStudentFees`, `rptFeesByClass`. Consistent prefixes make a project readable.
- Document the design: keep the table structures, the relationship diagram and the purpose of each query/form/report.
- Back up the database file regularly (a single `.accdb` file holds everything — losing it means losing the whole application).
- Compact and Repair periodically to prevent corruption and reduce file size.

PROPERTY **Exam tip: describing a database design**

When a question asks you to "describe the design of a database", present, in order: (1) the tables with their fields and primary keys; (2) the relationships with their cardinalities and referential integrity; (3) the queries answering the users' questions; (4) the forms for data entry; (5) the reports for printed output. This five-part structure mirrors the method above and earns full marks. (No proof required; the structure itself is examinable.)

EXERCISE 4 **Forms, reports and integration**

1. Give two differences between entering data in a form and in Datasheet View.
2. On a "New Payment" form, which control would you use for the `ClassID` field, and why?
3. In a report grouped by class, in which section do you place the expression that computes each class's total? And the school's grand total?
4. List, in order, the six steps for building a complete database application.

SOLUTION **Exercise 1**

1. A form shows one record at a time in a clear layout with labels and buttons, and can enforce validation (combo boxes, input masks, validation rules); Datasheet View shows many records as a raw grid with no guidance and no enforced validation.
2. A combo box fed by the `CLASS` table: the user picks an existing class instead of typing, which prevents invalid or inconsistent values.
3. The class total goes in the Group Footer (`=Sum([Amount])`); the grand total goes in the Report Footer.
4. Design tables and keys; set relationships with referential integrity; create the queries; build the forms; build the reports; test with realistic sample data.

5 INTEGRATION ACTIVITY

5.1 Integration Activity: Computerising School Fees Management

KEY POINTS **Targeted Competence**

Given the fee-management problem of a school, design and implement a complete relational database application (tables, relationships, queries, forms, reports) that produces the information the bursar needs, and justify the design decisions taken.

5.2 Aims of the Activity

This integration activity brings together everything you have learnt in this chapter: **table design**, **primary** and **foreign keys**, **relationships** with **referential integrity**, **selection**, **calculated** and **summary queries**, **forms** and **reports**. By the end of the activity, working in pairs, you should be able to:

- analyse a real administrative need and translate it into a small **relational database** of three related tables;
- choose appropriate **data types**, primary keys and **validation rules** for each field;
- enforce one-to-many relationships with referential integrity and justify the choice;
- write queries that answer concrete management questions, including **calculated fields** and **grouping** with aggregate functions (`SUM`, `COUNT`);
- build a user-friendly **data-entry form** (with a combo box and a subform) and a grouped, printable **report** with subtotals and a grand total;
- present and defend a design choice in front of an audience, as expected at the GCE Advanced Level practical examination.

5.3 Situation

The Collège Bilingue de Yaoundé is a fictitious private secondary school located at Nlongkak. Every term, the **bursar** (the économe) must know exactly who has paid school fees, how much each class has collected, and which families still owe money. Until now, everything has been recorded in a paper register: pages are lost, totals are recalculated by hand, and at the end of each term the bursar spends two full days preparing the report for the principal.

The principal has asked your ICT club to computerise the work. After interviewing the bursar, you collect the following facts:

- The school has several **classes**. Each class has a unique code, a name (e.g. Form 5 Science A), a level, and a fixed amount of fees due per student for the year.
- Each **student** has a unique matricule number, a full name, a sex, a date of birth, and belongs

to exactly one class.

- Fees are paid in **instalments**. Each payment concerns one student, is made on a given date, for a given amount in FCFA, and is attached to a term (1, 2 or 3). A student may make several payments per term.
- Annual fees per class (for this activity): Form 5 Science A: 150000 FCFA; Form 5 Arts B: 140000 FCFA; Lower Sixth Science: 165000 FCFA; Upper Sixth Science: 175000 FCFA.

The bursar's questions, which your application must answer at the click of a button, are:

1. "Give me the alphabetical list of the students of any class I choose."
2. "Give me the list of students who have paid **less than 50%** of their annual fees, with the balance remaining."
3. "How much has been collected per class and per term?"
4. "Which class has the highest total outstanding balance?"

He also wants a simple screen to record a payment by choosing the student's name from a drop-down list (he does not want to type matricules), a screen showing each student together with all his or her payments, and a printable end-of-term report, "Fees situation per class", grouped by class with a subtotal per class and a grand total for the school.

5.4 Tasks

Work in pairs on one computer, using **MS Access** or **LibreOffice Base**. Save your database as `CBY_Fees.accdb` (or `.odb`).

Part A — Tables and Relationships

1. Create the three tables below. For each field, choose a suitable data type and size, and write a short description.

Table	Fields (key underlined)	Suggested Types	Validation
CLASS	ClassID, ClassName, Level, FeesDue	AutoNumber / Short Text (50) / Short Text (30) / Currency	FeesDue > 0
STUDENT	Matricule, FullName, Sex, DateOfBirth, ClassID	Short Text (10) / Short Text (100) / Short Text (1) / Date/Time / Number (Long Integer)	Sex = "M" or "F"; DateOfBirth < `Date()`
PAYMENT	PaymentID, Matricule, DatePaid, Amount, Term	AutoNumber / Short Text (10) / Date/Time / Currency / Number (Byte)	Amount > 0; Term Between 1 And 3; DatePaid ≤ `Date()`

2. Explain why `Matricule` is a better primary key for STUDENT than `FullName`.
3. In the Relationships window, create the two **one-to-many** relationships (CLASS → STUDENT and STUDENT → PAYMENT) and tick Enforce Referential Integrity. State, in one sentence, what this forbids.

4. Enter the data: the 4 classes given above, at least 12 students (3 per class, with realistic Cameroonian names and birth dates), and at least 20 payments spread over the three terms, so that some students have paid everything and others less than half.

Part B — Queries

5. **Q1 — Class List.** Write a query that displays the full name, sex and class name of the students of one class (use a **parameter** `[Enter class name:]` on ClassName), sorted alphabetically on FullName.
6. **Q2 — Debtors.** Write a query showing, for each student: FullName, ClassName, FeesDue, a calculated field `TotalPaid: Sum(Amount)` and `Balance: FeesDue - Sum(Amount)`. Keep only students whose `TotalPaid` is less than `0.5 * FeesDue`. (Hint: this needs a `GROUP BY` on the student and a `HAVING` clause.)
7. **Q3 — Collections.** Write a summary query giving, per class and per term, the number of payments and the total amount collected (`GROUP BY ClassName, Term`).
8. **Q4 — Worst Class.** Using Q2's logic grouped per class (sum of balances), find the class with the highest outstanding balance.

Part C — Forms and Report

9. Build a **payment entry form** based on PAYMENT in which the student is chosen through a **combo box** displaying FullName but storing Matricule.
10. Build a **student form with a payments subform**, so that the bursar sees each student's payments on the same screen.
11. Produce the report "Fees situation per class": grouped by ClassName, showing each student's TotalPaid and Balance, a **subtotal** per class, and a **grand total** at the end. Add a title and the date.
12. Prepare a 3-minute demonstration: show the form, run Q2 and the report, and justify **one** design choice (key, data type or relationship).

> **WARNING** **Assessment Grid (20 Marks)**
>
> Correct keys and relationships: 4 pts — Queries Q1–Q4: 6 pts — Payment form with combo box and student/subform: 4 pts — Grouped report with subtotals and grand total: 4 pts — Presentation and justification: 2 pts. A database that opens but whose relationships are not enforced loses the 4 relationship points: test them by trying to enter a payment for matricule `ZZ999`.

5.5 Model Answer and Step-by-Step Guide

Part A — Tables and Relationships

METHOD **Creating Tables in Design View**

- Open the database, click `Create` → `Table Design`.
- For each field: enter Field Name, choose Data Type from the drop-down, set Field Size (for text/number), write a Description, and in the Field Properties pane (bottom) set `Validation Rule` and `Validation Text` where required.
- Set the Primary Key: select the field(s) and click `Primary Key` icon.
- Save with the table name (CLASS, STUDENT, PAYMENT).

DEFINITION **Data Type Choices**

- `AutoNumber` (Long Integer) for `ClassID` and `PaymentID`: the system guarantees a unique, immutable surrogate key.
- `Short Text` for `Matricule` (size 10), `ClassName` (50), `Level` (30), `FullName` (100), `Sex` (1). Text keys are stable and human-readable.
- `Currency` for `FeesDue` and `Amount`: fixed-point arithmetic avoids rounding errors on money.
- `Date/Time` for `DateOfBirth` and `DatePaid`: enables date arithmetic (age, term boundaries).
- `Number` (Long Integer) for `STUDENT.ClassID` (foreign key matching CLASS.ClassID); `Number` (Byte) for `Term` (values 1–3 only).

EXEMPLE **Validation Rules and Texts**

Field	Validation Rule	Validation Text
Sex	`In("M","F")`	`Enter M or F only`
FeesDue, Amount	`>0`	`Amount must be positive`
Term	`Between 1 And 3`	`Term must be 1, 2 or 3`
DateOfBirth	`<Date()`	`Birth date cannot be in the future`
DatePaid	`<=Date()`	`Payment date cannot be in the future`

Why Matricule and not FullName? A primary key must be **unique** and **stable**. Two students can share the same full name (e.g. two “Ngo Bassa Marie”), but the school issues each matricule once only. Names can change (marriage, correction), whereas a matricule never changes.

Relationships: Open `Database Tools` → `Relationships`. Add the three tables. Drag `CLASS.ClassID` onto `STUDENT.ClassID` → tick Enforce Referential Integrity → `Create`. Drag `STUDENT.Matricule` onto `PAYMENT.Matricule` → tick Enforce Referential Integrity → `Create`. Both are **one-to-many** (one class has many students; one student has many payments).

PROPERTY **Referential Integrity**

Enforcing referential integrity forbids:

- inserting a payment with a `Matricule` that does not exist in STUDENT;
- deleting a student who still has payments;
- deleting a class that still has students;
- changing a `ClassID` or `Matricule` in the parent table if matching child records exist (unless Cascade Update is also ticked).

Sample Data (extract):

ClassID	ClassName	Level	FeesDue (FCFA)
1	Form 5 Science A	Form 5	150000
2	Form 5 Arts B	Form 5	140000
3	Lower Sixth Science	Lower Sixth	165000
4	Upper Sixth Science	Upper Sixth	175000

Matricule	FullName	Sex	DateOfBirth	ClassID
CBY001	Abena Mballa Jean	M	12/03/2008	1
CBY002	Ngo Bassa Marie	F	25/07/2009	1
CBY003	Etoundi Onana Paul	M	03/11/2007	3
CBY004	Fotso Kamga Aline	F	18/01/2006	4
CBY005	Mbarga Essomba Pierre	M	30/09/2008	2
CBY006	Tchouakeu Diane	F	05/06/2009	2

PaymentID	Matricule	DatePaid	Amount (FCFA)	Term
1	CBY001	15/09/2024	100000	1
2	CBY001	10/01/2025	50000	2
3	CBY002	20/09/2024	40000	1
4	CBY004	05/10/2024	175000	1
5	CBY003	12/09/2024	80000	1
6	CBY003	15/01/2025	85000	2
7	CBY005	10/10/2024	70000	1
8	CBY006	22/09/2024	30000	1

Part B — Queries

METHOD Creating Queries in Design View

- Click `Create` → `Query Design`. Add required tables (CLASS, STUDENT, PAYMENT). Close `Show Table`.
- Drag fields to the grid. For calculated fields, type expression in a blank column: `TotalPaid: Sum([Amount])`.
- For grouping: click `Totals` (Σ) icon; a `Total` row appears. Choose `Group By`, `Sum`, `Count`, etc.
- For criteria on aggregates: use the `Having` row (appears when you right-click → `Show/Hide` → `Having`).
- For parameters: type `[Enter class name:]` in the `Criteria` row of ClassName.
- Switch to `SQL View` to see/check the generated SQL.

EXPERIMENT Key Queries in SQL

Q1 — Class List (Parameter Query):

```
SELECT S.FullName, S.Sex, C.ClassName
FROM CLASS C INNER JOIN STUDENT S ON C.ClassID = S.ClassID
WHERE C.ClassName = [Enter class name:]
ORDER BY S.FullName;
```

Q2 — Debtors (Less Than 50% Paid):

```
SELECT S.FullName, C.ClassName, C.FeesDue,
Sum(P.Amount) AS TotalPaid, C.FeesDue - Sum(P.Amount) AS Balance
FROM (CLASS C INNER JOIN STUDENT S ON C.ClassID = S.ClassID)
INNER JOIN PAYMENT P ON S.Matricule = P.Matricule
GROUP BY S.FullName, C.ClassName, C.FeesDue
HAVING Sum(P.Amount) < 0.5 * C.FeesDue;
```

Q3 — Collections per Class and per Term:

```
SELECT C.ClassName, P.Term, Count(*) AS NbPayments, Sum(P.Amount) AS Collected
FROM (CLASS C INNER JOIN STUDENT S ON C.ClassID = S.ClassID)
INNER JOIN PAYMENT P ON S.Matricule = P.Matricule
GROUP BY C.ClassName, P.Term
ORDER BY C.ClassName, P.Term;
```

Q4 — Worst Class (Highest Outstanding Balance):

```
SELECT C.ClassName, Sum(C.FeesDue - TotalPaid) AS TotalBalance
FROM (
  SELECT S.Matricule, C.ClassName, C.FeesDue, Sum(P.Amount) AS TotalPaid
  FROM (CLASS C INNER JOIN STUDENT S ON C.ClassID = S.ClassID)
  INNER JOIN PAYMENT P ON S.Matricule = P.Matricule
  GROUP BY S.Matricule, C.ClassName, C.FeesDue
) AS StudentBalances
GROUP BY C.ClassName
ORDER BY Sum(C.FeesDue - TotalPaid) DESC;
```

EXEMPLE **Interpreting Results with Sample Data**

Running Q2 returns, for example, Ngo Bassa Marie (TotalPaid 40000 FCFA, Balance 110000 FCFA), since 40000 FCFA < 0.5× 150000 FCFA = 75000 FCFA. Q3 returns one row per class per term, e.g. Upper Sixth Science — Term 1 — 6 payments — 620000 FCFA.

Part C — Forms and Report

METHOD **Payment Entry Form with Combo Box**

- Create → `Form Design`. Set `Record Source` to PAYMENT.
- Delete the default `Matricule` text box.
- Insert → `Combo Box`. Wizard: "I want the combo box to get the values from another table" → Next.
- Table: STUDENT; Fields: `Matricule`, `FullName` (in that order). Next.
- Sort by FullName ascending. Next.
- "Hide key column" → Next (this stores Matricule but shows FullName).
- "Store that value in this field" → Matricule. Finish.
- Add other fields (DatePaid, Amount, Term) from the Field List. Save as `frmPaymentEntry`.

METHOD **Student Form with Payments Subform**

- Create → `Form Wizard`. Tables/Queries: STUDENT (all fields except ClassID). Next.
- Choose `Form with subform(s)`. Next.
- Subform: PAYMENT (fields: DatePaid, Amount, Term). Next.
- Link: "Show PAYMENT for each record in STUDENT using Matricule". Next.
- Layout: Tabular for subform. Finish. Save as `frmStudentPayments`.

METHOD **Grouped Report: "Fees Situation per Class"**

- Create a query `qryFeesPerClass` joining CLASS, STUDENT, PAYMENT with calculated fields `TotalPaid: Sum([Amount])` and `Balance: [FeesDue]-[TotalPaid]`, grouped by ClassName, Matricule, FullName, FeesDue.
- Create → `Report Wizard`. Base on `qryFeesPerClass`. Select all fields. Next.
- Grouping: add `ClassName` to grouping levels. Next.
- Sort by FullName within group. Layout: Stepped, Orientation: Landscape. Finish.
- In `Report Design`:
  - In `ClassName Footer`: add text boxes with `=Sum([TotalPaid])` and `=Sum([Balance])` for class subtotals. Label them "Class Subtotal".
  - In `Report Footer`: add text boxes with same expressions for grand totals. Label "Grand Total".
  - Add a report header with title "Fees Situation per Class" and `=Date()` for today's date.
- Save as `rptFeesPerClass`.

Presentation (Example of Justification). "We enforced referential integrity on both relationships so that a payment can never point to a non-existent student; without it, a typing error on a matricule would silently create orphan payments and falsify the bursar's totals."

WARNING **Common Mistakes to Avoid**

- Do not put `Amount` in the STUDENT table (a student pays several times — that would repeat data and break normalisation).

- Do not use `FullName` as a key (not unique).
- Do not compute balances by hand — let the query do it with `SUM` and `GROUP BY`.
- Always test referential integrity with a deliberate error (e.g. enter payment for `ZZ999`) before the demonstration.
- Forgetting to hide the key column in the combo box (the bursar must see names, not matricules).
- Omitting the `HAVING` clause in Q2 (filtering on aggregate requires `HAVING`, not `WHERE`).
- Forgetting subtotals in the class footer and grand total in the report footer.

DID YOU KNOW? **Cameroon Context**

In Cameroon, many schools still manage fees with paper registers. The Ministry of Secondary Education (MINESEC) encourages digitalisation via the Plateforme Nationale de Gestion des Établissements. A well-designed Access/Base application like this one is a first step towards a networked solution (e.g. using MySQL/PostgreSQL with a web front-end) that could integrate with Mobile Money (MTN MoMo, Orange Money) for automatic payment recording — a real ICT project for Upper Sixth students!

6 METHODS AND WORKED EXAMPLES

7 CREATING TABLES AND RELATIONSHIPS

DID YOU KNOW? **Cameroon context**

In Cameroon, every secondary school manages thousands of student records. The Ministry of Secondary Education (MINESEC) requires accurate databases for examinations, orientation and statistics. A well-designed RDBMS prevents duplicate registrations, ensures each candidate has a unique index number, and allows quick generation of mark sheets for the GCE Board. The same principles apply to mobile money platforms (MTN MoMo, Orange Money) where each transaction must be linked to a valid subscriber.

7.1 Skill 1: Creating a well-structured table

DEFINITION **Table, Record, Field, Primary Key**

In a relational database, a table represents a single entity (e.g. Student). Each record (row) is one occurrence of that entity. Each field (column) is an attribute. The primary key (PK) is a field (or combination) that uniquely identifies every record; it cannot be null and cannot be duplicated.

METHOD **Designing and creating a table**

To create a table in an RDBMS (MS Access, LibreOffice Base, MySQL, etc.):

1. Analyse the situation: list the attributes describing the entity. Example for a Student: RegNo, FullName, Sex, DateOfBirth, ClassCode, FeesPaid.
2. Choose a data type for each field:
   - Short Text (names, codes) — specify Field Size.
   - Number: Integer, Long Integer, Double, etc.
   - Date/Time (dates, times).
   - Currency (monetary values, avoids floating-point errors).
   - Yes/No (Boolean).
   - AutoNumber (automatic unique identifier, often used as surrogate PK).

3. Choose the primary key: a field whose value is unique, never empty, and stable (e.g. RegNo). Avoid names, phone numbers or computed values as keys.
4. Set field properties to enforce data quality:
   - `Field Size` (limits length for text/numbers).
   - `Format` (display style, e.g. Short Date).
   - `Input Mask` (guides entry, e.g. `00/00/0000` for dates).
   - `Validation Rule` and `Validation Text` (e.g. `>=0` with text "Amount cannot be negative").
   - `Required = Yes` (not null).
   - `Default Value` (e.g. `0` for FeesPaid, `=Date()` for TransDate).
5. Save the table with a meaningful, space-free name using a prefix (e.g. `tblStudent`), then enter a few test records to verify constraints.

Reflexes: One table = one entity. Never store a calculated value (e.g. Age) that can be derived from a stored fact (DateOfBirth). Use Validation Rules to reject impossible data at entry time.

EXEMPLE **Data types and properties in Access vs. SQL**

Concept	MS Access / LibreOffice Base	Standard SQL (MySQL, PostgreSQL)
Auto-increment PK	AutoNumber	`AUTO_INCREMENT` (MySQL), `SERIAL` (PostgreSQL), `IDENTITY` (SQL Server)
Text (variable)	Short Text (max 255)	`VARCHAR(n)`
Long text	Long Text (Memo)	`TEXT`
Integer	Number (Long Integer)	`INT`
Decimal	Number (Double) / Currency	`DECIMAL(p,s)` / `NUMERIC`
Date	Date/Time	`DATE`, `DATETIME`, `TIMESTAMP`
Boolean	Yes/No	`BOOLEAN` / `TINYINT(1)`

WORKED EXAMPLE 5 **Creating the table tblStudent**

A school in Bamenda wants a database. Create the table `tblStudent` with fields: RegNo

(text, 8 characters, primary key), FullName (text, 40), Sex (M or F only), DateOfBirth (date), FeesPaid (currency, cannot be negative). State the data type, key and validation rules.

SOLUTION

Solution.

1. Fields and data types:

Field	Data type	Properties
RegNo	Short Text	Field Size = 8; Primary Key; Required = Yes
FullName	Short Text	Field Size = 40; Required = Yes
Sex	Short Text	Field Size = 1; Validation Rule: `In("M","F")`; Validation Text: "Enter M or F only"
DateOfBirth	Date/Time	Format = Short Date; Required = Yes
FeesPaid	Currency	Validation Rule: `>=0`; Default Value = 0; Required = Yes

2. Primary key: RegNo, because each student has a unique registration number; it is never null and never duplicated.
3. Justification of choices: Sex restricted to M/F prevents typing errors; the rule `>=0` on FeesPaid rejects impossible negative payments; DateOfBirth is stored rather than Age, because Age changes every year and can be computed in a query (see Skill 4).

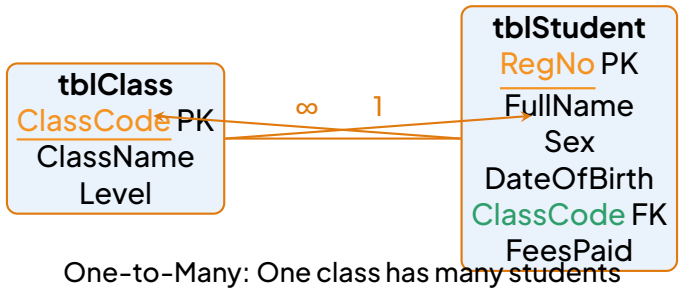
WARNING **Common GCE mistakes in table design**

- Using `Age` as a stored field instead of `DateOfBirth`.
- Choosing `FullName` or `PhoneNo` as primary key (not unique, can change).
- Forgetting `Required = Yes` on the primary key or on mandatory fields.
- Using `Number (Double)` for money — always use `Currency` or `DECIMAL` to avoid rounding errors (e.g. $0.1+0.2 \neq 0.3$ in binary floating point).
- Writing validation rule `="M" Or "F"` (wrong) instead of `In("M","F")` or `="M" Or ="F"`.

7.2 Skill 2: Creating relationships and enforcing referential integrity

DEFINITION **Foreign Key, Referential Integrity**

A foreign key (FK) is a field in a child table that references the primary key of a parent table. Referential integrity is a constraint that guarantees every non-null FK value matches an existing PK value in the parent table. It prevents orphan records (children pointing to non-existent parents).



One-to-Many: One class has many students

Entity-Relationship diagram for tblClass (1) — (∞) tblStudent

METHOD **Linking tables with relationships**

1. Identify the primary key of the "one" table (parent) and add a foreign key field of the exact same data type and size in the "many" table (child).
2. Open the Relationships window (Database Tools → Relationships), add both tables.
3. Drag the primary key onto the matching foreign key. The Edit Relationships dialog appears.
4. Tick Enforce Referential Integrity. Optionally tick:
   - Cascade Update Related Fields: if the PK changes, the FK updates automatically.
   - Cascade Delete Related Records: deleting a parent deletes its children (use with caution!).
5. Click Create. The join line shows 1 at the parent end and ∞ at the child end (one-to-many) or 1–1 (one-to-one).

Reflexes: A many-to-many relationship (e.g. Students ↔ Subjects) requires a junction table (e.g. `tblEnrollment`) containing the two PKs as FKs. Referential integrity forbids a child record pointing to a non-existent parent.

WORKED EXAMPLE 6 **Relating tblClass and tblStudent**

Given `tblClass`(ClassCode, ClassName, Level) and `tblStudent`(RegNo, FullName, ClassCode), (a) identify the primary and foreign keys; (b) describe how to create the relationship with referential integrity; (c) give two operations that referential integrity will reject.

SOLUTION

Solution.

1. Keys: Primary key of tblClass = ClassCode; Primary key of tblStudent = RegNo; Foreign key = ClassCode in tblStudent (references tblClass.ClassCode).
2. Type of relationship: One class contains many students → one-to-many (tblClass on the "1" side, tblStudent on the "∞" side).
3. Creation steps:
   (a) Open Relationships window; add tblClass and tblStudent.
   (b) Drag `tblClass.ClassCode` onto `tblStudent.ClassCode`.
   (c) In the dialog, tick Enforce Referential Integrity (and optionally Cascade Update Related Fields).
   (d) Click Create. The join line displays 1 near tblClass and ∞ near tblStudent.
4. Rejected operations (with referential integrity enforced):
   - Inserting a student with ClassCode "L6Z" if "L6Z" does not exist in tblClass.
   - Deleting (or changing the ClassCode of) a class that still has students, unless Cascade Delete/Update was ticked.
5. Benefit: The database stays consistent — no "orphan" student attached to a non-existent class.

WARNING **Referential integrity traps in exams**

- Data type mismatch: FK must be identical type and size to PK (e.g. both Short Text 10). A common error: PK is AutoNumber (Long Integer) but FK is Number (Integer) → relationship fails.
- Existing bad data: You cannot enforce referential integrity if the child table already contains FK values not present in the parent. Clean data first.
- Cascade Delete danger: Ticking Cascade Delete on a Class → Student relationship would delete all students of a class if the class is deleted. Usually do not tick Cascade Delete for core entities; instead, prevent deletion of parent if children exist.

8 QUERIES 1 — SELECTING AND FILTERING DATA

DID YOU KNOW? **Why queries matter**

A query is a saved question to the database. Unlike a filter, a query is an object you can reuse, combine, and base forms/reports on. In the GCE practical exam, you will be asked to write queries in Design View and often to provide the equivalent SQL statement. Master both views.

8.1 Skill 3: Simple select queries with criteria

METHOD **Building a SELECT query with conditions (Design View)**

1. Open Query Design; add the required table(s); close the Show Table dialog.
2. Drag desired fields from the table pane into the design grid columns.
3. Tick/untick Show to display or hide a field (a hidden field can still carry a criterion).
4. Write criteria on the `Criteria` row. Common patterns:
   - Exact match: `="M"`, `="L6A"`
   - Comparison: `>=50`, `<100`
   - Range: `Between 10 And 20` or `Between #1/1/2006# And #31/12/2006#` (dates use `#` `#`)
   - Pattern: `Like "A*"` (starts with A), `Like "*son"` (ends with son), `Like "?a*"` (second letter a)
   - Null check: `Is Null`, `Is Not Null`
5. Conditions on the same row = **AND** (all must be true). Conditions on different rows (Criteria row and `or` row) = **OR** (at least one true).
6. Choose Sort (Ascending/Descending) on any column.
7. Run the query (Datasheet View) to see results.

Key idea (SQL): `SELECT fields FROM table WHERE condition ORDER BY field;`

Design Grid: Female students with FeesPaid ≥ 50,000

Field	RegNo	FullName	FeesPaid	Sex
Table	tblStudent	tblStudent	tblStudent	tblStudent
Sort		Ascending		
Show	☰	☰	☰	
Criteria			>=50000	="F"
or				

Query Design Grid showing AND criteria on the same row

WORKED EXAMPLE 7 **Filtering students with criteria**

Using `tblStudent`(RegNo, FullName, Sex, DateOfBirth, FeesPaid, ClassCode): (a) display RegNo, FullName and FeesPaid of female students who paid at least 50 000 FCFA, sorted by name; (b) write the equivalent SQL; (c) display students who paid nothing or are in class “L6A”.

SOLUTION

Solution.

1. Design grid for (a):

Row	RegNo	FullName	FeesPaid	Sex
Field	RegNo	FullName	FeesPaid	Sex
Table	tblStudent	tblStudent	tblStudent	tblStudent
Sort		Ascending		
Show	☰	☰	☰	(unticked)
Criteria			>=50000	="F"
or				

   Both criteria sit on the same row (Criteria), so they are combined with AND: only girls who paid 50 000 FCFA or more appear.

2. SQL for (a) (Access/Jet dialect): `SELECT RegNo, FullName, FeesPaid FROM tblStudent WHERE Sex="F" AND FeesPaid>=50000 ORDER BY FullName;`

3. Query (c): Put `=0` on the Criteria row under FeesPaid, and `="L6A"` on the or row under ClassCode. Different rows mean OR, so a student is shown if FeesPaid = 0 or ClassCode = “L6A” (or both).

4. Warning: Writing both criteria on the same row in (c) would wrongly show only students of L6A who paid nothing (AND instead of OR).

WARNING **Criteria syntax traps**

- Dates: In Access SQL, dates must be surrounded by # #: `#12/31/2023#` (US format mm/dd/yyyy). In standard SQL use single quotes: `'2023-12-31'`.
- Text: Always in double quotes in Access: `"M"`. In standard SQL, single quotes: `'M'`.
- Wildcards: Access uses `*` (any chars) and `?` (one char). Standard SQL uses `%` and `_`.
- Null: Never write `= Null`; use `Is Null` or `Is Not Null`.

8.2 Skill 4: Calculated fields in queries

DEFINITION **Calculated Field**

A calculated field is a column whose value is computed at query run time from an expression involving other fields, operators, and functions. It is not stored in the table, avoiding redundancy and ensuring up-to-date results.

METHOD **Creating a calculated field in Design View**

1. In an empty column of the design grid, type: `NewFieldName: expression`.
2. Build the expression using:
   - Field names in square brackets: `[UnitPrice]`, `[DateOfBirth]`.
   - Arithmetic operators: `+ - * /` `^` (exponent), `Mod` (modulo).
   - Functions: `Year()`, `Month()`, `Day()`, `Date()` (current date), `Now()`, `DateDiff()`, `IIf()`, `Format()`, `Round()`, `Int()`.
3. Example — Age (approximate): `Age: Year(Date())-Year([DateOfBirth])`.
4. Better Age (exact): `Age: DateDiff("yyyy",[DateOfBirth],Date()) + IIf(Format(Date(),"mmdd")<Format([DateOfBirth],"mmdd"),-1,0)`.
5. A calculated field is computed at run time; it is never stored in the table.

Reflex: Use `IIf(condition, valueIfTrue, valueIfFalse)` for conditional results, e.g.

`Decision: IIf([Average]>=10,"Pass","Fail")`. **Nested IIf for multiple bands:** `Grade: IIf([Mark]>=70,"A",IIf([Mark]>=60,"B",IIf([Mark]>=50,"C","Fail")))`.

WORKED EXAMPLE 8 **Computing amounts with a calculated field**

A trader in Douala stores `tblSales`(SaleID, Product, UnitPrice, Quantity). (a) Create a query showing Product, UnitPrice, Quantity and a calculated field `Amount` = UnitPrice × Quantity. (b) Add a field `Category` displaying "Big sale" if Amount ≥ 100 000, otherwise "Small sale". (c) Explain why Amount must not be a table field.

SOLUTION

Solution.

1. Query (a): Add tblSales; drag Product, UnitPrice, Quantity; in the fourth column type: `Amount: [UnitPrice]*[Quantity]`
2. Query (b): In the fifth column type: `Category: IIf([UnitPrice]*[Quantity]>=100000,"Big sale","Small sale")`
3. Run: For a row UnitPrice = 25 000, Quantity = 6, the query displays Amount = 150 000 and Category = "Big sale".
4. Why not store Amount: It is derived data. If UnitPrice or Quantity is later corrected, a stored Amount would become wrong (redundancy and inconsistency). The calculated field always reflects current values, respecting the rule: store facts, compute results.

EXEMPLE **Useful Access functions for GCE**

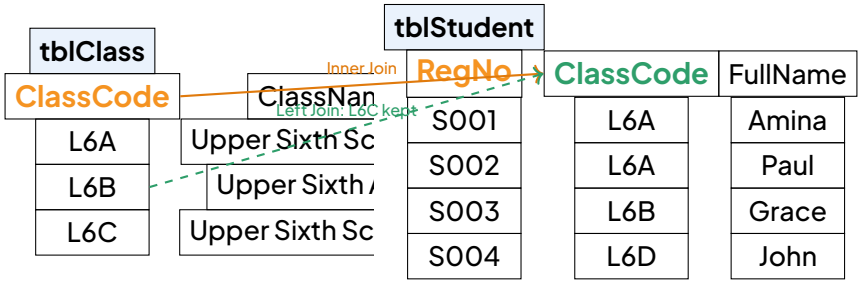
Function	Syntax	Example / Use
IIf	`IIf(cond, true, false)`	`IIf([Sex]="M","Male","Female")`
DateDiff	`DateDiff(interval, date1, date2)`	`DateDiff("d",[OrderDate],Date())` days since order
DateAdd	`DateAdd(interval, number, date)`	`DateAdd("m",6,[ExpiryDate])` add 6 months
Format	`Format(value, format)`	`Format([Amount],"#,##0.00")`
Year/Month/Day	`Year(date)` etc.	Extract parts of a date
Round	`Round(number, decimals)`	`Round([Avg],2)`
Nz	`Nz(value, ifNull)`	`Nz([Discount],0)` treat null as 0

9 QUERIES 2 — MULTI-TABLE AND SUMMARY QUERIES

9.1 Skill 5: Multi-table queries using joins

DEFINITION **Inner Join, Outer Join (Left/Right)**

An inner join returns only rows where the join condition matches in both tables. A left outer join (LEFT JOIN) returns all rows from the left (parent) table and matching rows from the right (child); non-matching child columns show Null. A right outer join does the opposite.



Inner Join: only L6A, L6B appear (L6C dropped, John dropped)
Left Join: L6C appears with Null student; John still dropped

Inner Join vs Left Outer Join illustration

METHOD **Querying several related tables**

1. Add all needed tables to the query; check that the join lines (relationships) appear. If a relationship does not exist, drag the key of one table onto the matching foreign key to create a temporary join.
2. Select fields from any of the tables: the RDBMS automatically matches each child record to its parent via the join (inner join by default: only matching pairs appear).
3. Add criteria and sorting as usual. Prefix field names with table name if ambiguous: `tblStudent.RegNo`.
4. To include parent records with no children (e.g. classes with no students), double-click the join line and choose the outer join option: "Include ALL records from `tblClass` and only those records from `tblStudent` where the joined fields are equal" (LEFT JOIN).

Reflex: The number of tables in the query must match the "path" of relationships between the fields used.

WORKED EXAMPLE 9 **A two-table query**

Given `tblClass`(ClassCode, ClassName) and `tblStudent`(RegNo, FullName, Sex, ClassCode): (a) list RegNo, FullName and ClassName of all students in the class whose ClassName is "Upper Sixth Science A", sorted by FullName; (b) write the SQL; (c) explain what changes if a class has no student yet.

SOLUTION

Solution.

1. Design: Add both tables; the join line ClassCode–ClassCode appears automatically (relationship created earlier). Drag RegNo, FullName from tblStudent and ClassName from tblClass. Under ClassName write the criterion `="Upper Sixth Science A"`; set Sort = Ascending under FullName.
2. SQL (Access):
```
SELECT tblStudent.RegNo, tblStudent.FullName,
tblClass.ClassName FROM tblClass INNER JOIN tblStudent ON tblClass.ClassCode
= tblStudent.ClassCode WHERE tblClass.ClassName="Upper Sixth Science A"
ORDER BY tblStudent.FullName;
```
3. How the join works: For each student row, the RDBMS looks up the row of tblClass having the same ClassCode and merges the two rows; the user sees the class name although the student table only stores the code.
4. Case (c): With the default inner join, an empty class does not appear at all, because it matches no student. To display it (with blank student columns), modify the join to a left outer join: "Include ALL records from tblClass and only those records from tblStudent where the joined fields are equal" (LEFT JOIN in SQL).

WARNING **Join mistakes in GCE**

- Missing join: Forgetting to add a table that lies on the path between two others (e.g. querying Student and Subject without Enrollment).
- Wrong join type: Using inner join when the question asks for "all classes, even those without students" → must use outer join.
- Ambiguous field names: Always prefix with table name in SQL: `tblStudent.ClassCode`. In Design Grid, Access adds the prefix automatically.
- Cartesian product: If you add two tables without a join line, you get every combination (cross join) → huge, wrong result.

9.2 Skill 6: Summary (aggregate) queries with GROUP BY

DEFINITION **Aggregate Functions, GROUP BY, HAVING**

Aggregate functions compute a single value from a set of rows: `Sum`, `Avg`, `Min`, `Max`, `Count`, `StDev`, `Var`. GROUP BY partitions rows into groups sharing the same value(s) in the grouping field(s); aggregates are computed per group. HAVING filters groups after aggregation (unlike WHERE which filters rows before grouping).

METHOD **Creating a Totals query (Design View)**

1. Create a select query with the needed tables and fields.
2. Click the Totals (Σ) button on the toolbar: a Total row appears in the grid.
3. For each column choose from the Total row dropdown:
   - Group By — the grouping field(s) (one row per distinct value).
   - Sum, Avg, Min, Max, Count, StDev, Var — aggregate functions.
   - Where — criterion applied before grouping; the field is not displayed in output.
   - Expression — for calculated fields that are then aggregated (e.g. `Sum([Price]*[Qty])`).
4. Rule: Every displayed field must be either a Group By field or an aggregate — never a raw detail field.
5. In SQL: `SELECT group, AggFunction(field) FROM table GROUP BY group HAVING condition;`

Reflex: `Count(*)` counts all rows in the group; `Count(field)` ignores Nulls in that field.

Totals Query Grid: Statistics per Class

Field	ClassCode	RegNo	FeesPaid	FeesPaid
Table	tblStudent	tblStudent	tblStudent	tblStudent
Total	Group By	Count	Sum	Avg
Show	☰	☰	☰	☰
Criteria				

Totals row with Group By, Count, Sum, Avg

WORKED EXAMPLE 10 **Statistics per class**

Using `tblStudent`(RegNo, FullName, Sex, FeesPaid, ClassCode): (a) produce, for each class, the number of students, the total fees collected and the average fee paid; (b) show only classes with more than 30 students; (c) write the SQL for (a) and (b).

SOLUTION

Solution.

1. Design grid (a): Add tblStudent, click Σ:

Row	ClassCode	RegNo	FeesPaid	FeesPaid
Field	ClassCode	RegNo	FeesPaid	FeesPaid
Total	Group By	Count	Sum	Avg
Show	☰	☰	☰	☰

   Rename columns for clarity: `NbStudents: RegNo, TotalFees: FeesPaid, AvgFee: FeesPaid`.

2. Result interpretation: The query returns one row per distinct ClassCode, e.g. L6A — 45 students — 2 250 000 FCFA — 50 000 FCFA average.

3. Condition (b): On the Count column (RegNo), add the criterion `>30` on the Criteria row. Because the condition applies to an aggregate result, it becomes a `HAVING` condition in SQL.

4. SQL: `SELECT ClassCode, Count(RegNo) AS NbStudents, Sum(FeesPaid) AS TotalFees, Avg(FeesPaid) AS AvgFee FROM tblStudent GROUP BY ClassCode HAVING Count(RegNo)>30;`

5. Warning: Writing `WHERE Count(RegNo)>30` is a classic GCE error: `WHERE` cannot use aggregate functions because it filters rows before they are grouped.

WARNING **GROUP BY / HAVING exam traps**

- WHERE vs HAVING: WHERE filters individual rows before grouping (e.g. `WHERE Sex="F"`). HAVING filters groups after aggregation (e.g. `HAVING Count(*)>30`). In Design View, putting a criterion on a `Group By` field puts it in WHERE; putting it on an aggregate column puts it in HAVING.
- Selecting non-grouped fields: `SELECT ClassCode, FullName, Count(*) ... GROUP BY ClassCode` is illegal (FullName not in GROUP BY and not aggregated). Access may return an arbitrary value; standard SQL rejects it.

- Count(*) vs Count(field): If a field has Nulls, `Count(field)` excludes them; `Count(*)` includes all rows.

10 FORMS, REPORTS AND INTEGRATION

DID YOU KNOW? **Forms vs Reports in real life**

At a Mobile Money kiosk in Buea, the agent uses a form on a tablet to quickly log each deposit/withdrawal (dropdown for customer, buttons for New/Save). At end of day, the agent prints a report grouped by transaction type with totals, to reconcile with the MTN statement. Forms are for interactive entry; reports are for formatted output.

10.1 Skill 7: Creating forms and reports

METHOD **Building a form (data entry / viewing)**

1. Select the source table or query, then use the Form Wizard: choose fields, layout (Columnar = one record at a time; Tabular = multiple records; Datasheet = spreadsheet-like), and style.
2. Refine in Design View:
   - Add a clear title in the Form Header.
   - Align controls (labels left, text boxes right), set consistent spacing.
   - Set Tab Order (View → Tab Order) for logical keyboard navigation.
   - Replace foreign key text boxes with Combo Boxes (drop-down lists):
     - Row Source: a query like `SELECT ClassCode, ClassName FROM tblClass ORDER BY ClassName;`
     - Bound Column = 1 (stores the code), Column Count = 2, Column Widths = "0cm;3cm" (hides code, shows name).
     - Limit To List = Yes (prevents typing invalid codes).
   - Add Command Buttons via Wizard: Add New Record, Save Record, Close Form, Print Report.
   - For one-to-many: create a Main Form / Subform (e.g. Class form with embedded Student subform).

METHOD **Building a report (printable output)**

1. Use the Report Wizard on a table or query; choose fields.
2. Define grouping levels (e.g. Group by ClassName) and sorting (e.g. Sort by FullName).
3. In Summary Options, tick aggregate functions (Sum, Count, Avg) for numeric fields; they appear in group footers and report footer.
4. Choose layout (Stepped, Block, Outline) and orientation (Portrait/Landscape).
5. Refine in Design View:
   - Adjust headers/footers: Report Header (title, date), Page Header (column titles), Group Header (group name), Detail (records), Group Footer (subtotals), Report Footer (grand totals), Page Footer (page numbers).
   - Use `=Count([RegNo])`, `=Sum([FeesPaid])` in footers.
   - Set `CanGrow/CanShrink` for text boxes to handle variable-length data.
   - Preview (Print Preview) before printing or exporting to PDF.

Reflex: Forms are for entering and viewing on screen; reports are for printing and presenting. Both are based on tables or queries — they store no data themselves.

WORKED EXAMPLE 11 **Form and report for the school database**

The bursar of a college in Yaoundé wants (a) a form to enter new students, choosing the class from a drop-down list; (b) a report listing students grouped by class, with the number of students and total fees per class, and a grand total at the end. Describe the construction steps.

SOLUTION

Solution.

1. Form (a):
   - Create → Form Wizard on `tblStudent`; select all fields; Columnar layout; name it `frmStudent`.
   - In Design View, replace the ClassCode text box by a Combo Box whose Row Source is `SELECT ClassCode, ClassName FROM tblClass ORDER BY ClassName;` — the bursar picks a class name, but the code is stored (Bound Column = 1). This prevents typing errors and guarantees referential integrity.
   - Add buttons via the Command Button Wizard: Add New Record, Save Record,

Close Form. Add a clear title "Student Registration" in the Form Header.

- Set Tab Order: RegNo → FullName → Sex → DateOfBirth → ClassCode (combo) → FeesPaid.

2. Report (b):
   - Create → Report Wizard on a query joining tblStudent and tblClass (to show class names); select ClassName, RegNo, FullName, FeesPaid.
   - Choose Group by ClassName; sort by FullName; in Summary Options tick Sum of FeesPaid and Count of RegNo; Stepped layout; name it `rptFeesByClass`.
   - In Design View, check the group footer contains `=Count([RegNo])` and `=Sum([FeesPaid])`, and the report footer contains `=Sum([FeesPaid])` for the grand total; add page numbers (`=[Page]`) and date (`=Date()`) in Page Footer.
   - Set `ForceNewPage` property of Group Footer to "After Section" if each class should start on a new page.
3. Result: Each class starts on a new band, lists its students alphabetically, and ends with its subtotal; the last page shows the school's grand total — a document the principal can print directly for a council meeting.

10.2 Skill 8: Integration — building a complete mini-application

METHOD **Integrating tables, queries, forms and reports (GCE Integration Task)**

For a complete small database application (typical 2-hour practical exam):

1. Design (paper first): Identify entities, draw the schema (tables, keys, relationships) on paper. Normalize to 3NF: no repeating groups, no partial dependencies, no transitive dependencies.
2. Build tables: Create tables with appropriate data types, primary keys, validation rules, required fields, default values. Save with `tbl` prefix.
3. Create relationships: Open Relationships window, add tables, drag PK to FK, enforce referential integrity (decide on Cascade Update/Delete).
4. Build queries: Create select, calculated, and summary queries as required by the scenario. Test each with known data; verify SQL view.
5. Create forms: Data-entry forms with combo boxes for FKs, command buttons for navigation/actions, clear titles, logical tab order. Main/Subform for 1:M.
6. Create reports: Grouped reports with sorting, subtotals in group footers, grand totals in report footer, page numbers, date, professional layout.

7. Verify end-to-end: Enter test records (including edge cases: boundary values, invalid data to check rejection, empty groups). Run all queries, open forms, preview reports. Confirm outputs match manual calculations.

Reflex: Save and name every object with a prefix (`tbl`, `qry`, `frm`, `rpt`) — examiners reward clear, consistent naming.

WORKED EXAMPLE 12 **Integration: a mobile-money agent database**

An MTN Mobile Money agent in Buea wants to manage transactions. Design a mini-application with: `tblAgent`(AgentID, AgentName, Town) — here a single agent, kept for extension; `tblCustomer`(PhoneNo, CustName, Town); `tblTransac`(TransID, TransDate, PhoneNo, TransType, Amount) where TransType is "Deposit" or "Withdrawal". (a) Draw the schema and state the relationships. (b) Give the query computing, per customer, the total deposited and total withdrawn (hint: use two queries or `IIf`). (c) Describe the data-entry form and the daily report.

SOLUTION

Solution.

1. Schema and relationships:

Table	Primary Key	Foreign Key	References
tblAgent	AgentID	—	—
tblCustomer	PhoneNo	—	—
tblTransac	TransID	PhoneNo	tblCustomer.PhoneNo

Relationship: tblCustomer (1) — (∞) tblTransac via PhoneNo (Short Text, same size). Enforce referential integrity so a transaction cannot reference an unknown phone number, and a customer with transactions cannot be deleted. TransType gets Validation Rule `In("Deposit","Withdrawal")`; Amount gets `>0`; TransDate gets Default Value `=Date()`.

2. Query (b): Create a totals query on tblCustomer INNER JOIN tblTransac:

Field	Total	Expression / Alias
CustName	Group By	—
Amount	Sum	`Deposits: Sum(IIf([TransType]="Deposit",[Amount],0))`
Amount	Sum	`Withdrawals: Sum(IIf([TransType]="Withdrawal",[Amount],0))`

SQL: `SELECT CustName, Sum(IIf(TransType="Deposit",Amount,0)) AS Deposits, Sum(IIf(TransType="Withdrawal",Amount,0)) AS Withdrawals FROM tblCustomer INNER JOIN tblTransac ON tblCustomer.PhoneNo=tblTransac.PhoneNo GROUP BY CustName;` The agent immediately sees each customer's position; the difference Deposits - Withdrawals can be added as a further calculated column: `Balance: [Deposits]-[Withdrawals]`.

3. Form (c): Form `frmTransac` on tblTransac, Columnar layout; PhoneNo as a combo box listing customers (Row Source: `SELECT PhoneNo, CustName FROM tblCustomer ORDER BY CustName;`); TransType as a combo box with values "Deposit";"Withdrawal"; TransDate with Default Value `=Date()`; buttons New Record, Save Record, Close Form. This makes entry fast and error-proof at the agent's kiosk.
4. Report: `rptDaily` based on a query filtering `TransDate = Date()`; group by TransType; sort by TransID; group footer with `=Count([TransID])` and `=Sum([Amount])`; report footer with the day's grand total. The agent prints it every evening for reconciliation with the MTN statement.
5. Verification: Enter two customers and five transactions by hand, compute the expected totals manually, and compare with the query and report outputs — the essential final step of any integration activity.

KEY POINTS **Key takeaways for GCE ICT23**

- Table design: One entity per table; PK unique/not null; choose correct data types (Currency for money); use validation rules; never store derived data (Age, Amount).
- Relationships: PK/FK same type/size; enforce referential integrity; 1:M is standard; M:N needs junction table.
- Queries: Design Grid ↔ SQL equivalence. Criteria row = AND; or row = OR. Dates in # #. Wildcards * ?.
- Calculated fields: `Name: Expression` with [Field] syntax; IIf for conditions; computed at runtime.
- Joins: Inner (default) vs Outer (Left/Right) for "all parents even without children".
- Aggregates: Totals button (Σ); Group By + aggregates; HAVING for group conditions (not WHERE).
- Forms: Combo boxes for FKs (Row Source query, Bound Column, Limit To List); command buttons; Main/Subform for 1:M.
- Reports: Grouping, sorting, summary options (Sum/Count/Avg), group/report footers for subtotals/grand totals.

- Integration: Design → Build → Query → Interface → Verify. Naming convention `tbl/qry/frm/rpt`.

EXERCISE 5 **Practice: School Library**

A school library in Douala needs a database. Entities: Book(ISBN, Title, Author, Category, CopiesTotal), Member(MemberID, Name, Class, Phone), Loan(LoanID, ISBN, MemberID, DateOut, DateDue, DateReturned).

1. Draw the ER diagram (identify PKs, FKs, relationship types).
2. Write the validation rule for Category (values: "Novel", "Science", "History", "Other").
3. Create a query showing overdue books (DateDue < Date() AND DateReturned Is Null) with Member Name, Book Title, DaysOverdue (calculated).
4. Create a summary query: per Category, number of loans and average days overdue (for returned books only).
5. Sketch the loan entry form (combo boxes for Member and Book) and the overdue report (grouped by Class).

SOLUTION **Exercise: School Library**

1. ER Diagram:
   - Book (1) — (∞) Loan via ISBN.
   - Member (1) — (∞) Loan via MemberID.
   - PKs: Book.ISBN, Member.MemberID, Loan.LoanID.
   - FKs: Loan.ISBN (ref Book), Loan.MemberID (ref Member).
2. Category validation: `In("Novel","Science","History","Other")`.
3. Overdue query (Design Grid):

Field	Member.Nam	Book.Title	DateDue	DateReturnec	DaysOverdue
Table	Member	Book	Loan	Loan	Loan
Total					
Sort			Ascending		
Show	☰	☰	☰		☰
Criteria			<Date()	Is Null	

Calculated field: `DaysOverdue: DateDiff("d",[DateDue],Date())`. SQL: `SELECT Member.Name, Book.Title, Loan.DateDue, DateDiff("d",Loan.DateDue,Date()) AS DaysOverdue FROM (Member INNER JOIN Loan ON Member.MemberID=Loan.MemberID) INNER JOIN Book ON Loan.ISBN=Book.ISBN WHERE Loan.DateDue<Date() AND Loan.DateReturned Is Null ORDER BY Loan.DateDue;`

4. Summary query: Base on a query joining Book, Loan (filter DateReturned Is Not Null). Group By Category. Count(LoanID) as NbLoans. Avg(DateDiff("d",DateDue,DateReturned)) as AvgDaysOverdue.

5. Form: frmLoan on tblLoan. Combo for MemberID (Row Source: MemberID, Name), Combo for ISBN (Row Source: ISBN, Title). DateOut default =Date(). Buttons: New, Save, Close. Report: rptOverdue on overdue query. Group by Member.Class; sort by Member.Name. Group footer: Count of loans. Report footer: Grand count.

11 EXERCISES

11.1 I check my knowledge

EXERCISE 6 **MCQ on RDBMS fundamentals**

For each question, choose the correct answer.

1. In a relational database, a primary key is:
   A. A field that can contain duplicate values.
   B. A field (or combination of fields) that uniquely identifies each record in a table.
   C. A field that links two tables together.
   D. A field that is automatically indexed but not unique.
2. The relationship between a `Student` table and a `Class` table where one class contains many students but each student belongs to only one class is:
   A. One-to-One
   B. Many-to-Many
   C. One-to-Many
   D. Many-to-One
3. Referential integrity ensures that:
   A. No duplicate records exist in a table.
   B. A foreign key value must match an existing primary key value in the referenced table (or be Null).
   C. All fields are filled before saving a record.
   D. Data is backed up automatically.
4. In Query Design view, placing criteria on the same row (Criteria row) for different fields creates a logical:
   A. OR condition
   B. AND condition
   C. NOT condition
   D. XOR condition

EXERCISE 7 **True/False with justification**

State whether each statement is True or False. Justify your answer briefly.

1. A table in a relational database can have more than one primary key.
2. A foreign key field must have the same name as the primary key it references.
3. In a one-to-many relationship, the one side is the table containing the foreign key.
4. A `SELECT` query modifies the data stored in the underlying tables.
5. The `GROUP BY` clause is used to sort the result set of a query.

EXERCISE 8 **Key definitions**

Define the following terms in the context of a Relational Database Management System (RDBMS):

1. Entity
2. Attribute
3. Relationship (and give the three main types)
4. Referential Integrity
5. Query
6. Form
7. Report

EXERCISE 9 **Direct application: Data types and field properties**

For each field described below, choose the most appropriate data type (Text, Number, Date/Time, Currency, Yes/No, OLE Object, Hyperlink, Attachment, AutoNumber) and suggest one useful field property (e.g., Field Size, Format, Input Mask, Validation Rule, Required, Indexed).

1. `StudentID`: Unique identifier automatically generated for each new student.
2. `DateOfBirth`: Student's date of birth.
3. `AnnualFees`: School fees amount in FCFA (e.g., 150000).
4. `IsBoarder`: Indicates whether the student is a boarder (Yes/No).

5. `Photo`: Passport photograph of the student.
6. `Email`: Student's email address.
7. `RegistrationDate`: Date and time the record was created.

11.2 I practise

EXERCISE 10 **Structured question: Library database**

Consider the following two tables for a school library:

LIBRARY	LOAN
BookID (PK)	LoanID (PK)
Title	BookID (FK)
Author	StudentName
Category	DateOut
Publisher	DateBack

1. Identify the primary key and foreign key in each table.
2. State the type of relationship between `LIBRARY` and `LOAN`. Draw the relationship line with cardinalities (1, ∞).
3. Write the criteria (in Query Design grid style) to list all books that are currently on loan (i.e., not yet returned). Assume `DateBack` is empty (Null) for books not returned.
4. Explain why referential integrity prevents deleting a record from `LIBRARY` if there are matching records in `LOAN`.

EXERCISE 11 **Query design: School fees database**

A school fees database contains the tables:

- `STUDENT(StudentID, FirstName, LastName, Gender, Form, ClassName)`
- `PAYMENT(PaymentID, StudentID, DatePaid, AmountPaid)`
- `FEESTRUCTURE(Form, Term, AmountDue)`

Design a query (show the design grid: Field, Table, Sort, Show, Criteria, Or) that displays the `FirstName`, `LastName`, `ClassName`, and the balance owed (AmountDue − Sum of AmountPaid) for all Form 5 girls who still owe more than 20 000 FCFA, sorted by balance descending. You may assume a calculated field `Balance: [AmountDue] -`

`Sum([AmountPaid])`. Indicate clearly the grouping and criteria rows.

EXERCISE 12 **SQL reading exercise (enrichment)**

Study the following SQL statement and explain in plain English what the query returns.

```
SELECT ClassName, SUM(AmountPaid) AS TotalCollected
FROM STUDENT INNER JOIN PAYMENT ON STUDENT.StudentID = PAYMENT.StudentID
WHERE Form = 'Upper Sixth'
GROUP BY ClassName
HAVING SUM(AmountPaid) > 500000;
```

EXERCISE 13 **Data types and properties: Hospital patient table**

A hospital database needs a `PATIENT` table with the following fields. For each, choose the most appropriate data type and justify your choice. Also suggest one validation rule or input mask where relevant.

1. `PatientNo` (unique identifier, e.g., P00123)
2. `FullName`
3. `DateOfBirth`
4. `Weight` (in kg, one decimal place)
5. `Admitted` (Yes/No)
6. `Photo` (scan of patient ID card)
7. `Notes` (long medical history text)

11.3 I prepare for the GCE

EXERCISE 14 **Troubleshooting a query**

A user creates a query to find all `Form 5` students in `Class 'Science A'`. The query uses two criteria: `Form = "Form 5"` and `ClassName = "Science A"`. The query returns no records, even though the user knows such students exist. Identify three possible causes for this problem and state the correction for each.

EXERCISE 15 **Choosing the right database object**

For each of the following tasks, state whether a Table, Query, Form, or Report is the most appropriate object to use. Justify your choice.

1. Entering a new student's details (Name, Form, Date of Birth, Parent Contact).
2. Printing end-of-term fee statements for all students in Form 5, showing each student's payments and balance.
3. Checking on screen the current balance of a single student whose ID is known.
4. Producing an alphabetical class list for the notice board, showing Student Name, Form, and Class.
5. Recording a new payment made by a student (Date, Amount, Receipt Number).

EXERCISE 16 **Design exercise: Yaoundé Cybercafé**

A cybercafé in Yaoundé wants to track customers, computers, and browsing sessions.

1. Propose three tables with appropriate fields. Indicate primary keys (PK) and foreign keys (FK). Choose suitable data types.
2. Draw the relationship diagram (describe the links and cardinalities: 1:∞, etc.).
3. List two queries the owner would need (describe in words what each query returns).
4. Describe one report the owner would find useful (layout and grouping).

12 SOLUTIONS TO THE EXERCISES

SOLUTION **Exercise 1 — MCQ on RDBMS fundamentals**

1. Correct answer: B. A **primary key** is a field (or combination of fields) that uniquely identifies each record in a table. It cannot contain duplicate values (so A is wrong) and cannot be Null. Option C describes a foreign key, and option D describes an ordinary index.
2. Correct answer: C. One class contains many students, and each student belongs to one class: this is a **one-to-many** (1:∞) relationship, with `Class` on the "one" side and `Student` on the "many" side.
3. Correct answer: B. **Referential integrity** guarantees that a foreign key value must either match an existing primary key value in the referenced (parent) table or be Null. It prevents "orphan" records, e.g. a loan record pointing to a book that does not exist.
4. Correct answer: B. Criteria placed on the same Criteria row of the Query Design grid are combined with a logical **AND**: a record must satisfy all the criteria to be selected. Criteria on different rows (Criteria and Or rows) create an OR condition.

SOLUTION **Exercise 2 — True/False with justification**

1. False. A table can have only one primary key. That primary key may, however, be composite (made of several fields combined). Other unique fields can exist, but they are called candidate keys or unique indexes, not primary keys.
2. False. A foreign key must have a compatible data type (and size) with the primary key it references, but the names may differ. For example, `LOAN.BookID` could reference `LIBRARY.BookCode`. Using the same name is only a good naming convention.
3. False. It is the opposite: in a one-to-many relationship, the foreign key is placed in the table on the many side. Example: `ClassID` (PK of `Class`, the one side) is stored as a foreign key inside `Student` (the many side).
4. False. A `SELECT` query only reads and displays data; it never modifies the underlying tables. Queries that modify data are action queries: `UPDATE`, `DELETE`, `APPEND` (make-table).
5. False. `GROUP BY` groups records sharing the same value so that aggregate functions (`SUM`, `COUNT`, `AVG`…) can be computed per group. Sorting is done with `ORDER BY` (or the Sort row in the design grid).

SOLUTION **Exercise 3 — Key definitions**

1. Entity: a real-world object, person, event or concept about which data is stored (e.g. Student, Book, Payment). In a relational database, an entity is implemented as a table.
2. Attribute: a single property or characteristic of an entity (e.g. `LastName`, `DateOfBirth` of a Student). An attribute is implemented as a field (column) of a table.
3. Relationship: an association between two entities (tables), usually implemented by placing a foreign key in one table that references the primary key of another. The three main types are: one-to-one (1:1), one-to-many (1:∞) and many-to-many (∞:∞, implemented through a junction/linking table).
4. Referential Integrity: a set of rules enforced by the RDBMS ensuring that relationships between tables remain valid: a foreign key value must match an existing primary key value in the parent table (or be Null), and a parent record cannot be deleted (or its key changed) while matching child records exist, unless cascading is explicitly enabled.
5. Query: a question or request posed to the database to retrieve, filter, calculate, combine or modify data. Queries are built in Design view (QBE grid) or written in SQL.
6. Form: a screen object used to enter, edit and display data one record at a time in a user-friendly layout, with controls such as text boxes, combo boxes and buttons.
7. Report: a formatted, printable object used to present data (often grouped, sorted and with totals) for output on paper or PDF, e.g. fee statements, class lists.

SOLUTION **Exercise 4 — Direct application: Data types and field properties**

Field	Data type	Suggested property
StudentID	AutoNumber	Indexed: Yes (No Duplicates); set as Primary Key
DateOfBirth	Date/Time	Format: Short Date; Input Mask: 99/99/0000
AnnualFees	Currency	Format: Currency (FCFA); Validation Rule: >= 0
IsBoarder	Yes/No	Format: Yes/No; Default Value: No
Photo	OLE Object (or Attachment)	— (no size property needed)
Email	Hyperlink (or Text)	Validation Rule: Is Null Or Like "*@*.*"
RegistrationDate	Date/Time	Format: General Date; Default Value: Now()

Justifications: AutoNumber guarantees a unique, automatically generated identifier; Currency avoids rounding errors on money amounts; Yes/No stores a Boolean efficiently; OLE Object/Attachment stores binary images; Now() automatically stamps the creation date and time.

SOLUTION **Exercise 5 — Structured question: Library database**

1. Keys: In LIBRARY: primary key = BookID; no foreign key. In LOAN: primary key = LoanID; foreign key = BookID (it references LIBRARY.BookID).
2. Relationship: One book can appear in many loans (over time), but each loan concerns exactly one book: a one-to-many relationship, LIBRARY(1) — (∞)LOAN.



One-to-many relationship between LIBRARY and LOAN.

3. Criteria for books currently on loan: use the LOAN table, field DateBack, Criteria row: Is Null. To list the book titles, join LIBRARY to LOAN and show Title, StudentName, DateOut.

Field	Title	StudentName	DateBack
Table	LIBRARY	LOAN	LOAN
Show	✓	✓	✓
Criteria			Is Null

4. Referential integrity and deletion: if a book record in `LIBRARY` were deleted while `LOAN` still contains rows referencing its `BookID`, those loan rows would become orphans pointing to a non-existent book, and the loan history would be meaningless. With referential integrity enforced, the RDBMS refuses the deletion (unless "Cascade Delete Related Records" has been deliberately ticked, in which case the matching loans are deleted together with the book).

SOLUTION **Exercise 6 — Query design: School fees database**

The query joins `STUDENT` — `PAYMENT` (on `StudentID`) and `STUDENT` — `FEESTRUCTURE` (on `Form`). Totals (Group By) must be activated. The design grid is:

Field	FirstName	LastName	ClassName	Gender	Form	Balance: [AmountDue] – Sum([AmountPaid])
Table	STUDENT	STUDENT	STUDENT	STUDENT	STUDENT	
Total	Group By	Group By	Group By	Group By	Group By	Expression
Sort						Descending
Show	✓	✓	✓			✓
Criteria				"F"	"Form 5"	> 20000

Explanations:

- All displayed descriptive fields are set to Group By; the calculated field `Balance` is an Expression combining the amount due with the aggregate `Sum([AmountPaid])`.
- The criteria `"F"` (Gender), `"Form 5"` (Form) and `> 20000` (Balance) are all on the same Criteria row, so they are combined with AND: only Form 5 girls owing more than 20 000 FCFA appear.
- `Gender` and `Form` are not shown (Show unchecked) but still filter the records.
- Sorting Descending on `Balance` lists the largest debtors first.

WARNING **Common mistake**

Do not put the balance condition in a `WHERE`-style criterion on `Sum([AmountPaid])` directly: an aggregate result must be tested after grouping (in SQL this is the `HAVING` clause). In the design grid, placing the criterion on the calculated Expression column handles this correctly.

SOLUTION **Exercise 7 — SQL reading exercise (enrichment)**

Reading the statement clause by clause:

- `FROM STUDENT INNER JOIN PAYMENT ON …`: the query combines each student with his/her payment records by matching `StudentID`.
- `WHERE Form = 'Upper Sixth'`: only Upper Sixth students are kept.
- `GROUP BY ClassName`: the remaining records are grouped class by class.
- `SELECT ClassName, SUM(AmountPaid) AS TotalCollected`: for each class, the total of all payments is computed and displayed under the heading `TotalCollected`.
- `HAVING SUM(AmountPaid) > 500000`: only classes whose total exceeds 500 000 FCFA are displayed.

Plain English: "For each Upper Sixth class, give the class name and the total amount of fees collected, but show only the classes that have collected more than 500 000 FCFA." Note the difference: `WHERE` filters individual rows before grouping, while `HAVING` filters groups after aggregation.

SOLUTION **Exercise 8 — Data types and properties: Hospital patient table**

1. `PatientNo`: Text (Field Size 6). Although it looks like an identifier, it contains a letter (P00123), so Number/AutoNumber is impossible. Property: Input Mask `>P00000` and Indexed: Yes (No Duplicates); make it the Primary Key.
2. `FullName`: Text (Field Size 50–100). Names are alphanumeric and of variable length. Property: Required: Yes.
3. `DateOfBirth`: Date/Time. Enables date calculations such as the patient's age. Property: Format Short Date; Validation Rule `<= Date()` (a birth date cannot be in the future).
4. `Weight`: Number, Field Size Single (or Double), Format with one decimal place. A nu-

meric type is required for calculations (BMI, dosage). Validation Rule: `> 0 And < 500`.

5. `Admitted`: Yes/No. A Boolean flag (admitted or not). Property: Default Value: No.
6. `Photo`: OLE Object (or Attachment). Stores the scanned ID card image. Attachment is preferred in modern Access because it compresses files.
7. `Notes`: Memo / Long Text. Medical histories exceed the 255-character limit of a Text field and need long, unformatted text.

SOLUTION **Exercise 9 — Troubleshooting a query**

Three plausible causes (any three accepted at the GCE):

1. Criteria placed on different rows (OR instead of AND). If `"Form 5"` is on the Criteria row and `"Science A"` on the Or row, the query returns all Form 5 students plus all Science A students — or, conversely, a contradictory combination returns nothing. Correction: place both criteria on the same Criteria row to create the AND condition.
2. Spelling / spacing mismatch in the criteria. The stored value may be `"Science A"` (double space), `"science a"` (different case in some settings) or `"Sciences A"`, while the criterion says `"Science A"`; exact-match text criteria then select nothing. Correction: check the actual values in the table, or use a wildcard criterion such as `Like "Science*"`.
3. Missing or broken join / wrong table. If the query includes two tables (e.g. `STUDENT` and `PAYMENT`) without a proper join line, or the join is on the wrong field, the combination may produce an empty result set. Correction: create the relationship (join) on the matching key fields, e.g. `StudentID`.

Other acceptable answers: the Form field stores a code (e.g. `F5`) rather than the text "Form 5"; extra spaces in the stored data (fix with `Trim` or corrected data entry); criteria typed with the wrong data type (quotes around a number).

SOLUTION **Exercise 10 — Choosing the right database object**

1. Form. A data-entry form presents one student at a time with labelled text boxes, combo boxes (Form, Class) and validation, making entry fast and reducing errors. (Data could be typed directly into the table, but a form is the professional choice.)
2. Report. Fee statements are printed output, grouped per student with payment details, subtotals and balances — exactly what a grouped report (with a subreport or grouping levels) produces.

3. Query. A select query with a parameter criterion on `StudentID` (e.g. `[Enter Student ID:]`) returns the current balance on screen instantly; no printing or data entry is needed.
4. Report. A class list for the notice board is printed output, sorted alphabetically and possibly grouped by Form/Class — a report with a Sort on `LastName` (Ascending).
5. Form. Recording a new payment is data entry into the `PAYMENT` table; a form (possibly a subform of the student form) with fields Date, Amount and Receipt Number is appropriate.

Examiner expectation: the justification must mention the purpose (entry → Form; printed/grouped output → Report; on-screen retrieval/calculation → Query; permanent storage → Table).

SOLUTION **Exercise 11 — Design exercise: Yaoundé Cybercafé**

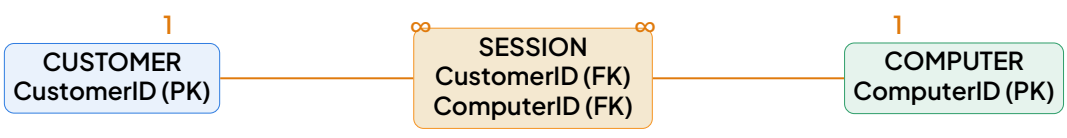
Indicative mark scheme (20 marks): tables and keys 8, data types 3, relationship diagram 4, queries 3, report 2.

1. Proposed tables:

Table	Fields (data types)
`CUSTOMER`	CustomerID (AutoNumber, PK), FullName (Text, 50), Phone (Text, 15), Email (Text/Hyperlink)
`COMPUTER`	ComputerID (AutoNumber, PK), Label (Text, e.g. PC-01), Status (Yes/No: working)
`SESSION`	SessionID (AutoNumber, PK), CustomerID (Number, FK), ComputerID (Number, FK), StartTime (Date/Time), EndTime (Date/Time), AmountCharged (Currency)

   `SESSION` is the junction table: it resolves the many-to-many link between customers and computers (a customer uses many computers over time; a computer is used by many customers).

2. Relationship diagram: `CUSTOMER` (1) — (∞) `SESSION` (∞) — (1) `COMPUTER`.

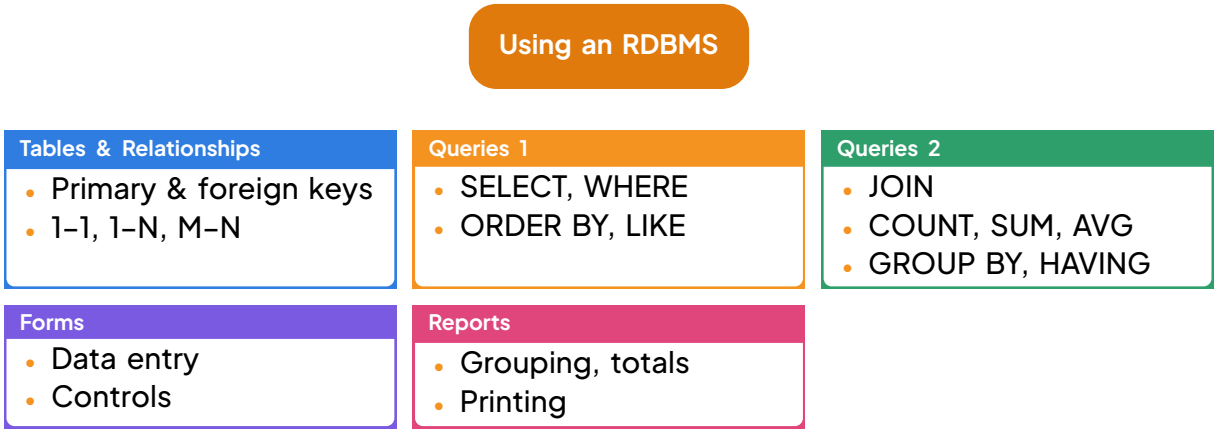


Two one-to-many relationships: one customer has many sessions; one computer hosts many sessions.

3. Two useful queries (any two):
   - Daily income: total of `AmountCharged` grouped by date (`DateValue([StartTime])`) — shows how much was earned each day.
   - Busy computers: count of sessions grouped by `ComputerID` for the current month — identifies the most (and least) used machines.
   - Customer history: all sessions of a given customer (parameter query on `CustomerID`) with total time and total spent.
4. A useful report: a Daily Sales Report, grouped by date, then by computer within each date, listing each session (customer, start/end time, amount) with a subtotal per computer and a grand total per day in the group footer. This lets the owner see at a glance which machines earn the most and control the cashier's receipts.

Examiner expectations: PK/FK clearly labelled; FK data type must match the PK it references (Number ↔ AutoNumber); the junction table `SESSION` must carry both foreign keys; cardinalities shown as 1 and ∞; queries described by what they return, not just their titles.

13 SUMMARY SHEET



Mind map of the chapter: from table design to printed reports.

FICHE BILAN **L'essentiel à connaître pour l'examen**

Key definitions

- A **relational database management system** (RDBMS) is software that stores data in **tables** (relations) made of **records** (rows) and **fields** (columns), and manages links between tables. Examples: Microsoft Access, MySQL, PostgreSQL, Oracle.
- A **primary key** is a field (or set of fields) that uniquely identifies each record of a table; it can never be empty (no `NULL`) and never duplicated. An `AutoNumber` field is often used when no natural unique field exists.
- A **foreign key** is a field in one table that refers to the primary key of another table; it is what creates a **relationship** between the two tables.
- **Referential integrity**: a foreign key value must either match an existing primary key value in the referenced table or be empty. It prevents "orphan" records (e.g. a payment attached to a student who does not exist) and blocks the deletion of a record that is still referenced elsewhere.
- A **query** is a question asked to the database, written in **SQL** (Structured Query Language) or built visually in a design grid (QBE: Query By Example). A **form** is a screen object used for entering, editing and displaying data; a **report** is a formatted, printable output used to present data on paper or PDF.

Relationships to know

- **One-to-many** (1–N): the most common. One student, many fee payments. The foreign key sits on the "many" side (the payment table carries the student's code).

- **One-to-one** (1–1): rare; used to split a very large table or to isolate confidential fields (e.g. medical details of a student).
- **Many-to-many** (M–N): cannot be stored directly; it is implemented through a **junction table** (also called a linking or bridge table) containing the two primary keys as foreign keys (e.g. Students–Courses via an Enrolment table). Each M–N link becomes two 1–N links.

SQL essentials

- Selection: `SELECT field1, field2 FROM Table WHERE condition ORDER BY field ASC;` (use `DESC` for descending order; `SELECT *` returns all fields).
- Comparison and logical operators: `=`, `<`, `>`, `<=`, `>=`, `<>` (not equal), `AND`, `OR`, `NOT`, `BETWEEN ... AND ...`, `IN (...)`.
- Pattern matching: `LIKE 'A*'` in Microsoft Access, `LIKE 'A%'` in standard SQL (names beginning with A). Text values are written between single quotes; numbers are written without quotes.
- Multi-table: `SELECT ... FROM A INNER JOIN B ON A.code = B.code;` which returns only the records whose key values match in both tables.
- Summary (aggregate) functions: `COUNT(*)`, `SUM(field)`, `AVG(field)`, `MIN(field)`, `MAX(field)`, combined with `GROUP BY field` to compute one result per group, and filtered after grouping by `HAVING condition`.
- Action queries: `INSERT INTO Table VALUES (...);`, `UPDATE Table SET field = value WHERE condition;`, `DELETE FROM Table WHERE condition;`.

Methods

- To design a database: identify the entities (objects of the real world), list their fields, choose a suitable data type for each field (Short Text, Number, Date/Time, Currency, Yes/No, AutoNumber), assign a primary key to every table, then draw the relationships and enforce referential integrity.
- To build a query in a design grid: add the table(s), drag the required fields into the grid, type the criteria on the Criteria row (criteria on the same row are combined with AND; on different rows with OR), choose the sort order, untick Show for fields used only as criteria, then run the query and check the result.
- To create a form: base it on a table or query, arrange the controls (text boxes, labels, combo boxes, command buttons), then test data entry.
- To create a report: base it on a table or query, group and sort the records (e.g. by class), add totals in the group footer, then preview before printing.

Common mistakes

- Forgetting the primary key, or choosing a field that can repeat (e.g. a surname) as key.

- Putting the foreign key on the wrong side of a 1–N relationship: it must be on the "many" side.
- Confusing `WHERE` (filters individual records before grouping) with `HAVING` (filters the groups after `GROUP BY`).
- Using `*` instead of `%` as wildcard in standard SQL, or quoting numbers as if they were text.
- Running `UPDATE` or `DELETE` without a `WHERE` clause: every record of the table is then modified or erased.
- Trying to store a many-to-many link inside a single table (repeating groups) instead of creating a junction table.
- Confusing a form (data input on screen) with a report (formatted output for printing).

GCE tips

- Read the question twice: "list only the names of students in Form 5" means a `SELECT` with a `WHERE`, not a report.
- When asked for a total, average or count per class, per town or per month, think immediately of `GROUP BY` plus an aggregate function.
- In practical sessions, always enforce referential integrity and test it by trying to enter an invalid foreign key value: the RDBMS must refuse it.
- State clearly the difference between a form (input) and a report (output): examiners ask it very often.
- When writing SQL by hand, end each statement with a semicolon and check that every text value is quoted and every field name is spelt exactly as in the table.

Signed • OnBuch+
ONBUCH+ TEACHING TEAM
FOLLOWS THE OFFICIAL MINESEC SYLLABUS

OnBuch+